YouTube DataOps & Analytics Platform (2026 Edition)

Enterprise Architektur-Handbuch, Technisches Repertorium & Deep-Dive Dokumentation

Von Bare-Metal-Hadoop zu Serverless Cloud Lakehouse, Apache Iceberg V2 & DataOps

---

Dokumenten-Typ:	Enterprise Architecture Reference, Technical Repertorium & Operations Manual
Autor & Lead Architect:	Bernd Weltler
Aktuelle Version:	Release `v2.1.0` (Serverless Cloud & Observability) — inkl. `v1.0.0` & `v2.0.0` Historie
Status:	Produktionsreif (100% Deployed, Getestet & Verifiziert)
GCP Projekt-ID:	`cloudanalytics-514018` (Region: `europe-west3` Frankfurt / Location: EU)
Service Account:	`cloudanalytics-514018@cloudanalytics-514018.iam.gserviceaccount.com`
GitHub Repository:	`https://github.com/BHW80/DE_Capstone_2026.git`
Technologie-Stack:	Google Cloud Pub/Sub (v2.1 Serverless Streaming), Apache Kafka KRaft (v1.0/v2.0), Apache Spark 3.5, Apache Iceberg V2, dbt Core 1.13, Google BigQuery, Terraform 1.17 (7 GCP Ressourcen), Google Cloud Workflows, Grafana Cloud, GitHub Actions

Executive Scope: Dieses Handbuch dient als lückenloses Referenzwerk, Betriebshandbuch und Repertorium für die vollständige Modernisierung der YouTube Trending Video Data Pipeline. Es dokumentiert jede Architekturentscheidung (ADRs), die Migration von fragilen On-Premises-Altsystemen auf serverloses Cloud-Streaming (Google Cloud Pub/Sub in `v2.1.0` als Ablösung von lokalem Kafka) und Open Table Formats (Apache Iceberg V2), den Aufbau eines dimensionalen Data Warehouse nach Ralph Kimball (SCD-2 via dbt Core), das Infrastructure-as-Code-Design mit Terraform 1.17 (7 GCP-Ressourcen), den 0-€ Kostenvorteil serverloser Cloud Workflows, das cloud-native Observability-Konzept via Grafana Cloud sowie die GitHub Actions CI/CD-Automatisierung. Darüber hinaus stellt es die Prozesse für alle drei Releases (`v1.0.0`, `v2.0.0`, `v2.1.0`) in drei dedizierten Abschnitten gegenüber, beleuchtet kritisch 8 reale Risiken und Schwachstellen im Enterprise-Umfeld (inkl. TB/PB-Mitigationen) und bereitet mit 25 technischen Deep-Dive Interview-Fragen (Q&As) auf Senior/Lead-Architektur-Reviews vor.

Inhaltsverzeichnis

1. Management Summary & Strategisches Leitbild	Seite 3
2. Die 3 Release-Prozesse im Detail (v1.0.0, v2.0.0, v2.1.0) & Notwendigkeits-Matrix	Seite 4
3. End-to-End Systemarchitektur, Medallion Lakehouse & Repository-Struktur	Seite 9
4. Streaming Ingestion & Resilienz (Cloud Pub/Sub v2.1 vs. Kafka KRaft, DLQ & SCD-2)	Seite 12
5. Ephemerer Compute & Open Table Format (Apache Iceberg V2)	Seite 14
6. Cloud Data Warehouse & Kimball Star-Schema (dbt Core & SCD-2)	Seite 16
7. DataOps Quality Gates: 15 dbt Tests & In-Workflow Validierung	Seite 18
8. Infrastructure as Code (Terraform 1.17 Cloud Provisioning — 10 GCP Ressourcen & Stored Procedures)	Seite 19
9. Serverlose Orchestrierung (Google Cloud Workflows & Pattern B Stored Procedures)	Seite 21
10. Cloud-Native Observability (Grafana Cloud & GCP Monitoring - Weg 1)	Seite 24
11. DataOps CI/CD Pipelines, GitOps & Release-Management	Seite 25
12. Architectural Decision Records (ADRs 001–007) & Kritische Enterprise-Risikoanalyse	Seite 27
13. Deppensicheres Betriebshandbuch (9-Schritte-Anleitung & Playbooks)	Seite 30
14. Data Engineering Interview Masterclass & Technical Defense (25 Deep-Dive Q&As)	Seite 34
15. FinOps, Kosten-Engineering & Always-Free-Tier Kapazitätsplanung	Seite 39

1. Management Summary & Strategisches Leitbild

Die moderne Datenlandschaft verlangt von Analytics-Plattformen heute weit mehr als das bloße Ausführen statischer SQL-Skripte oder nächtlicher Batch-Jobs. Unternehmen stehen vor der Herausforderung, kontinuierlich eintreffende Datenströme fehlertolerant aufzunehmen, fehlerhafte Events ohne Datenverlust zu isolieren, Datenänderungen historisch lückenlos nachzuvollziehen (SCD-2) und strenge analytische Qualitätsgarantien durchzusetzen — und das bei minimalen Betriebskosten.

Das vorliegende Projekt **YouTube DataOps & Analytics Platform (2026 Edition)** demonstriert die vollständige Transformation eines wartungsintensiven universitären Legacy-Systems (Post Graduate Program in Data Engineering, Purdue University / Simplilearn 2021) über drei sauber getrennte Evolutions-Releases (`v1.0.0`, `v2.0.0`, `v2.1.0`) in eine cloud-native, serverlose Enterprise-Lösung.

Kernmetriken & Business Impact im Vergleich:

Dimension	Legacy-System (2021)	DataOps Platform (2026 v2.1)	Business & Tech Impact
Monatliche Infrastrukturkosten	Hohe VM-Kosten oder unkalkulierbare Cloud-Cluster	**0,00 € laufende Idle-Kosten** (Cloud Pub/Sub, Cloud Workflows & GCP Free Tier)	100% Kostensicherheit; keine 24/7 Kafka-VMs oder Composer-Gebühren (~300 €/Monat gespart)
Streaming & Fehlertoleranz (DLQ)	Lokaler Kafka+ZooKeeper; Pipeline bricht bei Fehlern ab	**Serverless Cloud Pub/Sub** (`topic-youtube-ingest` + `topic-youtube-dlq`)	Zero Data Loss ohne lokalen Broker: 444 Gutdaten + 56 korrupte Records (~10%) sauber getrennt
Transaktionssicherheit (ACID)	Flache Parquet-/CSV-Dateien (Gefahr von Teilabbrüchen)	**100% GCS Cloud Lakehouse & Apache Iceberg V2** (`gs://lakehouse-youtube-...`)	0 lokale Parquets/Icebergs abseits der Ausgangslage; alle Daten & Snapshots direkt im GCS Bucket
Historisierung & Auditfähigkeit	SCD-1 (Alte Werte werden blind überschrieben)	**SCD-2 Historisierung** via Cloud Workflows (`LEAD()`) & dbt Snapshots	Lückenlose Nachvollziehbarkeit mutierter Videotitel (`2kyS6SvSYSE`) über alle 3 Versionen
Orchestrierung & Quality Gates	Keine automatisierten Datenprüfungen	**8-Stufen End-to-End Cloud Workflow + 15 dbt Data Tests**	Cloud Workflow steuert GCS->BQ Load, Staging, Dims, Fact & FK-Gate komplett serverlos (`run_pipeline.ps1`)
Deployment & Reproduzierbarkeit	Manuelle Windows-Klick-Anleitungen & Batch-Dateien	**Terraform 1.17 IaC + GitHub Actions CI/CD**	7 GCP-Ressourcen (inkl. Pub/Sub Topics & Subscriptions) in 60s reproduzierbar bereitgestellt

Architektur-Prämisse:
Die fundamentale Prämisse dieser Plattform lautet: **Zero Idle Cost bei maximaler Produktionshärte**. In Release `v2.1.0` wurden sowohl der lokale Kafka-Broker (durch Google Cloud Pub/Sub) als auch der lokale Airflow-Cluster (durch Google Cloud Workflows) und der lokale Prometheus-Stack (durch Grafana Cloud) vollständig serverlos in die Cloud verlagert.

2. Die 3 Release-Prozesse im Detail (v1.0.0, v2.0.0, v2.1.0) & Notwendigkeits-Matrix

Dieses Kapitel stellt die exakten End-to-End-Prozessabläufe und Architekturen der drei Projektreleases (`v1.0.0`, `v2.0.0` und `v2.1.0`) in **drei eigenständigen Abschnitten** gegenüber und arbeitet präzise heraus, wie jede Evolutionsstufe die Engpässe ihres Vorgängers auflöst.

2.1 Ausgangslage: Die 2021 Legacy-Architektur (Historische Schmerzpunkte)

- **Windows OS-Inkompatibilität:** Lokale Hadoop/HDFS-Aufrufe scheiterten an Windows-Pfaden und erforderten inoffizielle `winutils.exe`-Binaries.
- **ZooKeeper als Single Point of Failure:** Separate ZooKeeper-Prozesse führten unter Windows zu Verbindungsabbrüchen, Session-Timeouts und blockierten Kafka-Brokern.
- **Fehlende Dead Letter Queue (DLQ):** Ein einziger korrupter Datensatz mit nicht-numerischen `views` brachte die gesamte Pipeline zum Absturz.
- **SCD-1 Blindüberschreibung:** Änderungen an Videotiteln wurden ohne Historie überschrieben. Punktgenaue Analysen vor/nach Titeländerungen waren unmöglich.

2.2 Abschnitt 1: Prozess & Architektur in Release v1.0.0 (Core Foundation — Lokale Container-Pipeline)

Architektur-Ziel von `v1.0.0`: Vollständige Eliminierung der Windows-Hadoop-Abhängigkeiten (`winutils.exe`) und des fehleranfälligen ZooKeeper-Dienstes durch eine rein lokale, containerisierte Medallion-Pipeline auf Basis von Docker Compose, Apache Kafka (KRaft) und PySpark.

Der End-to-End-Prozess in Release `v1.0.0` (4 Prozessschritte):

- **Prozessschritt 1 (Lokales Kafka KRaft Streaming & DLQ):** Start des lokalen Docker-Containers `de_kafka` (`start_kafka.ps1`) im ZooKeeper-losen KRaft-Modus auf `127.0.0.1:9092`. Der Producer (`run_producer.ps1` -> `src/producer.py`) liest `us_videos_mini_500.csv`, injiziert 10% korrupte Records (`views = 'CORRUPT_INVALID_NUMBER'`) und routet valide Daten in das lokale Kafka-Topic `topic_ingest` sowie fehlerhafte Daten in `topic_err`. Anschließend schreiben `run_bronze.ps1` (`src/consumer_bronze.py`) und `run_dlq.ps1` (`src/consumer_dlq.py`) lokale Parquet-Dateien nach `data/bronze/` bzw. `data/quarantine/`.
- **Prozessschritt 2 (Lokale PySpark Silver & Gold Transformation auf Parquet):** Der ephemere Spark-Container (`run_silver.ps1` -> `src/transform_silver.py`) führt die Typkonvertierung und den Broadcast-Join mit `US_category_id.json` aus und speichert flache Parquet-Dateien unter `data/silver/`. Danach berechnet `run_gold.ps1` (`src/transform_gold.py`) einfache Aggregationen (`channel_stats`, `category_stats`) als flache Parquet-Dateien unter `data/gold/`.
- **Prozessschritt 3 (Lokale Orchestrierung via Airflow 2.9):** Ein lokaler Airflow- und PostgreSQL-Container-Stack (`start_airflow.ps1` -> `docker/docker-compose-airflow.yml`) führt den DAG `dags/youtube_medallion_dag.py` auf Port `8085` aus.
- **Prozessschritt 4 (Lokales Container-Monitoring):** Lokale Prometheus- und Grafana-Container (`start_monitoring.ps1` -> `docker/docker-compose-monitoring.yml`) überwachen die Container auf den Ports `9090` und `3000`.

Architektonische Grenzen von `v1.0.0` (Warum `v2.0.0` nötig wurde): Reine Parquet-Ordner bieten keine ACID-Transaktionen und kein Time-Travel; das flache Spark-Gold-Skript überschreibt Dimensionen blind nach **SCD-1** (keine Titel-Historie); und der gleichzeitige Betrieb von Kafka, Spark, Airflow, Postgres, Prometheus und Grafana belastet den lokalen Host mit 6+ Container-Daemons.

2.3 Abschnitt 2: Prozess & Architektur in Release v2.0.0 (Lakehouse & Kimball Cloud DWH — Hybrid-Cloud)

Architektur-Ziel von `v2.0.0`: Einführung eines echten Open Table Formats (**Apache Iceberg V2**) im Silver-Layer sowie Ablösung der starren Spark-Gold-Skripte durch ein cloud-basiertes **Kimball Star-Schema mit SCD-2 Historisierung** in **Google BigQuery** und **dbt Core**.

Der End-to-End-Prozess in Release `v2.0.0` (4 Prozessschritte):

- **Prozessschritt 1 (Lokales Kafka KRaft Streaming mit deterministischer SCD-2 Mutation):** Weiterhin über lokalen Docker-Kafka (`start_kafka.ps1`, `run_producer.ps1`, `run_bronze.ps1`, `run_dlq.ps1`), jedoch erweitert um die deterministische 3-stufige Titel- und Tag-Mutation für das Ziel-Video `2kyS6SvSYSE`.
- **Prozessschritt 2 (Silver Transformation in Apache Iceberg V2):** `run_silver.ps1` (`src/transform_silver.py`) bindet die `iceberg-spark-runtime-3.5` ein und schreibt die bereinigten Daten mit ACID-Garantie und Snapshot-Isolation in die Iceberg-Tabelle `local.silver.videos` (`data/iceberg_warehouse/silver/videos/`) sowie als Staging-Parquet nach `data/silver/`.
- **Prozessschritt 3 (Cloud-Brücke nach Google BigQuery):** `load_to_bigquery.ps1` (`src/cloud_loader_bigquery.py`) lädt die Silver-Parquet-Daten idempotent (`WRITE_TRUNCATE`) in die Tabelle `silver_videos` im BigQuery-Dataset `youtube_analytics` (Location `EU`).
- **Prozessschritt 4 (dbt Core Kimball Star-Schema, SCD-2 Snapshot & 15 Tests):** `run_dbt.ps1 snapshot` führt `dim_video_snapshot` (`strategy='check'`) aus und historisiert alle 3 Titelversionen von `2kyS6SvSYSE` mit `valid_from`/`valid_to`. `run_dbt.ps1 run` baut das dimensionale Star-Schema (`dim_video`, `dim_category`, `dim_date` und `fact_video_metrics` mit Point-in-Time Join). `run_dbt.ps1 test` validiert 15 Schema- und Foreign-Key-Tests (`PASS=15`). Damit wird `run_gold.ps1` (`src/transform_gold.py`) in `v2.0.0` obsolet.

Architektonische Grenzen von `v2.0.0` (Warum `v2.1.0` nötig wurde): Das Streaming erforderte weiterhin einen lokal laufenden Docker-Kafka-Broker; Cloud-Ressourcen mussten noch manuell angelegt werden (kein Terraform IaC); und es fehlte eine serverlose Cloud-Orchestrierung sowie ein cloud-natives Monitoring ohne lokale Container.

2.4 Abschnitt 3: Prozess & Architektur in Release v2.1.0 (Serverless Cloud & Observability — Aktuelle Produktion)

Architektur-Ziel von `v2.1.0`: Vollständige Eliminierung aller dauerhaft laufenden Infrastruktur-Container und jeglicher lokaler Zwischenspeicherung (0 lokale Parquets/Icebergs abseits der Ausgangslage `data/raw` & `data/sample`!) durch **Terraform IaC (7 GCP-Ressourcen)**, **Serverless Google Cloud Pub/Sub**, **100% Cloud Lakehouse Storage im GCS Bucket**, **End-to-End-Steuerung über Google Cloud Workflows (alle 8 DWH-Schritte)**, **Grafana Cloud Weg 1** und **GitHub Actions CI/CD**.

Der End-to-End-Prozess in Release `v2.1.0` (5 Prozessschritte bzw. 1-Klick via `run_pipeline.ps1`):

- **Prozessschritt 1 (Deklaratives Cloud-Provisioning via Terraform 1.17):** `run_infra.ps1 apply` rollt vollautomatisch **7 GCP-Ressourcen** in `cloudanalytics-514018` aus: GCS Lakehouse Bucket `gs://lakehouse-youtube-cloudanalytics-514018`, BigQuery Dataset `youtube_analytics`, die beiden Pub/Sub Topics `topic-youtube-ingest` und `topic-youtube-dlq`, die Pull-Subscriptions `sub-youtube-ingest` und `sub-youtube-dlq` sowie den 8-stufigen End-to-End Cloud Workflow `youtube-dataops-pipeline`.
- **Prozessschritt 2 (Serverless Streaming & DLQ direkt in GCS — 0 lokale Dateien!):** `run_pubsub_producer.ps1` (`src/cloud_pubsub_producer.py`) streamt die 500 Events aus `us_videos_mini_500.csv` per HTTPS/REST an Cloud Pub/Sub (`topic-youtube-ingest` + 10% DLQ an `topic-youtube-dlq`). `run_pubsub_bronze.ps1` (`src/cloud_pubsub_consumer_bronze.py`) und

`run_pubsub_dlq.ps1` (`src/cloud_pubsub_consumer_dlq.py`) ziehen die Nachrichten aus `sub-youtube-ingest` bzw. `sub-youtube-dlq`, erzeugen die Parquet-Dateien im Arbeitsspeicher (`io.BytesIO()`) und schreiben sie **direkt in den GCS-Bucket** (`gs://.../bronze/` und `gs://.../quarantine/`).

- **Prozessschritt 3 (Cloud-to-Cloud Silver Transformation & Apache Iceberg V2 im GCS Bucket):** `run_silver.ps1` (`src/cloud_silver_gcs.py`) liest die Bronze-Parquets direkt aus `gs://.../bronze/`, führt Typkonvertierung und Kategorie-Join (`US_category_id.json`) durch und schreibt sowohl `gs://.../silver/silver_videos.parquet` als auch das komplette **Apache Iceberg V2 Warehouse** (`gs://.../iceberg_warehouse/silver/videos/`) direkt in den GCS-Bucket. Lokal verbleiben ausschließlich `data/raw` und `data/sample`.
- **Prozessschritt 4 (Vollständige End-to-End-Steuerung über Google Cloud Workflows):** `run_workflow.ps1` (`src/cloud_workflow_runner.py`) startet den Cloud Workflow `youtube-dataops-pipeline` in `europe-west3`, der **den gesamten DWH-Ablauf in 8 Schritten serverlos in GCP steuert**: 1. `LOAD DATA OVERWRITE` direkt aus `gs://.../silver/*.parquet` nach BigQuery `silver_videos`, 2. Row-Count-Check, 3. Aufbau `stg_videos`, 4. Aufbau `dim_category`, 5. Aufbau `dim_date`, 6. Aufbau der SCD-2 Dimension `dim_video` (inkl. aller 3 Titelversionen von `2kyS6SvSYSE` per `LEAD()`-Window-Timeline), 7. Aufbau `fact_video_metrics` (Point-in-Time Join) und 8. Foreign-Key Quality Gate (`0 quality errors detected`). Ergänzend validiert `run_dbt.ps1 test` alle 15 dbt-Schema-Tests (`PASS=15`).
- **Prozessschritt 5 (Zero-Footprint Observability in Grafana Cloud & CI/CD):** Grafana Cloud liest über das native Google Cloud Monitoring Plugin (`stackdriver`) live die Metriken von Cloud Workflows, BigQuery und Cloud Pub/Sub aus (`dashboards/youtube_dataops_monitoring.json`). Über den Master-Runner `run_pipeline.ps1` laufen die Schritte 2 bis 4 vollautomatisch mit einem einzigen Befehl durch.

2.5 Evolutions- und Notwendigkeits-Matrix über alle Releases

Kriterium	2021 Legacy	v1.0.0 (Foundation)	v2.0.0 (Lakehouse)	v2.1.0 (Aktuell)
Bereitstellung	Bare Metal Windows	Docker Compose	Docker + Python `.venv`	**Terraform 1.17 (7 Res.)**
Streaming Engine	Kafka + ZooKeeper	Lokales Kafka KRaft	Lokales Kafka KRaft	**Serverless Cloud Pub/Sub**
Resilienz & DLQ	Keine (Jobabbruch)	Kafka `topic_err`	Kafka `topic_err`	**Pub/Sub** `topic-youtube-dlq`
Storage Format	Flache CSV-Dateien	Snappy Parquet	**Apache Iceberg V2**	**Iceberg V2 + GCS Bucket**
Datenmodellierung	Unstrukturiert	Flache Spark Gold Marts	**Kimball Star-Schema**	**Kimball + SCD-2 Marts**
Historisierung	Keine (SCD-1)	Keine (SCD-1)	**SCD-2 dbt Snapshots**	**Point-in-Time Join DWH**
Quality Gates	Keine	Basis Row-Count	**15 dbt Data Tests**	**15 Tests + Workflow Gate**
Orchestrierung	Manuelle Batch-Skripte	Airflow 2.9 (Docker)	PowerShell CLI Runner	**Serverless Cloud Workflows**
Observability	Keine	Lokaler Prometheus/Grafana	Lokaler Container-Stack	**Grafana Cloud (Weg 1)**
CI/CD & Releases	Keine	Git Tag `v1.0.0`	Git Tag `v2.0.0`	**GitHub Actions (`v2.1.0`)**

Release	Zwingend notwendig (Must-Have)	Optional / Durch Cloud abgelöst (Nicht notwendig)
Release v1.0.0 ***(Core Foundation)***	• Docker Desktop & Docker Compose • Lokaler Apache Kafka KRaft (`start_kafka.ps1`) • Lokaler Kafka Producer & Consumer (`run_producer.ps1`, `run_bronze.ps1`, `run_dlq.ps1`) • Ephemerer Docker PySpark (`run_silver.ps1`, `run_gold.ps1`) • Lokaler Airflow 2.9 Stack (`start_airflow.ps1`) • Lokaler Prometheus & Grafana Stack (`start_monitoring.ps1`)	• **Kein GCP Cloud Account nötig** (100% lokal) • **Kein Cloud Pub/Sub, kein BigQuery, kein dbt Core** • **Kein Apache Iceberg V2** (nur lose Parquet-Dateien) • **Kein Terraform, keine Cloud Workflows, keine Grafana Cloud**
Release v2.0.0 ***(Lakehouse & Kimball Marts)***	• Python `.venv` + GCP Projekt & Service Account Key • Lokaler Apache Kafka KRaft (`start_kafka.ps1`, `run_producer.ps1`, `run_bronze.ps1`, `run_dlq.ps1`) • Apache Iceberg V2 im Spark-Container (`run_silver.ps1`) • BigQuery Loader (`load_to_bigquery.ps1`) • dbt Core Kimball Star-Schema, SCD-2 Snapshot & 15 Tests (`run_dbt.ps1`)	• **Altes Spark Gold Script (`src/transform_gold.py`, `run_gold.ps1`):** Obsolet! Durch dbt Kimball Marts in BigQuery abgelöst. • **Noch nicht vorhanden:** Terraform, Cloud Pub/Sub, Cloud Workflows, Grafana Cloud, GitHub Actions CI/CD.

Release v2.1.0 *(Serverless Cloud & Observability)* [AKTUELLE PRODUKTION]	• Terraform 1.17 (`run_infra.ps1`: 7 GCP-Ressourcen) • **1-Klick Master-Runner (`run_pipeline.ps1`)** • **Serverless Pub/Sub -> GCS (`gs://...`)** (`run_pubsub_producer.ps1`, `run_pubsub_bronze.ps1`, `run_pubsub_dlq.ps1`) • **Cloud Silver & Iceberg V2 in GCS** (`run_silver.ps1` / `cloud_silver_gcs.py`) • **8-Stufen End-to-End Cloud Workflow** (`run_workflow.ps1`) & 15 dbt Tests (`run_dbt.ps1 test`) • **Grafana Cloud (Weg 1)** & GitHub Actions CI/CD	• **Lokale Parquet-/Iceberg-Ordner (`data/bronze`, `quarantine`, `silver`, `iceberg_warehouse`, `gold`): ELIMINIERT!** Alles liegt zu 100% im GCS Bucket (`gs://lakehouse-youtube-...`), lokal bleibt nur die Ausgangslage (`raw` & `sample`). • **Lokaler Docker-Kafka-Stack (`start_kafka.ps1`, `run_producer.ps1` etc.): DURCH CLOUD PUB/SUB ABGELÖST!** • **Lokaler Airflow (`start_airflow.ps1`) & Monitoring (`start_monitoring.ps1`): DURCH CLOUD WORKFLOWS & GRAFANA CLOUD ABGELÖST!**

Produktions-Leitlinie v2.1.0:
Wichtige Betriebs-Leitlinie für Release v2.1.0 (Aktuell): Für den regulären Betrieb in `v2.1.0` müssen **weder lokales Kafka (`start_kafka.ps1`), noch lokaler Airflow (`start_airflow.ps1`), noch lokales Monitoring (`start_monitoring.ps1`) gestartet werden!** Das Streaming läuft serverlos über **Google Cloud Pub/Sub** (`run_pubsub_producer.ps1` -> `run_pubsub_bronze.ps1` / `run_pubsub_dlq.ps1`), die Orchestrierung über **Cloud Workflows** (`run_workflow.ps1`) und das Monitoring über **Grafana Cloud**.

3. End-to-End Systemarchitektur, Medallion Lakehouse & Repository-Struktur

Die Plattform folgt dem **Medallion Lakehouse Architektur-Muster** (Bronze -> Silver -> Gold), kombiniert dieses jedoch mit modernem In-Warehouse ELT (Extract-Load-Transform) und dimensionaler Star-Schema-Modellierung nach Ralph Kimball.

3.1 Die 6 Datenfluss-Schichten im Detail (Stand Release v2.1.0)

1. Raw Landing & Serverless Streaming Ingestion (Google Cloud Pub/Sub in v2.1 / Kafka KRaft in v1.0–v2.0):

In Release `v2.1.0` liest der serverlose Producer (`src/cloud_pubsub_producer.py` via `run_pubsub_producer.ps1`) das Sample `data/sample/us_videos_mini_500.csv` (500 Events) und publiziert diese per HTTPS/REST direkt an **Google Cloud Pub/Sub**: **444 valide Datensätze** fließen in `topic-youtube-ingest`, während **56 synthetisch korrumpierte Datensätze (~10% Fehlerrate)** direkt in das Dead-Letter-Queue-Topic `topic-youtube-dlq` geroutet werden. Für das Ziel-Video `2kyS6SvSYSE` mutiert der Producer deterministisch Titel und Tags über drei Trending-Tage. (Alternativ steht für Offline-Tests aus `v1.0`/`v2.0` weiterhin der lokale Kafka-KRaft-Producer `src/producer.py` bereit).

2. Bronze & Quarantäne Persistenz (In-Memory Parquet direkt in den GCS Bucket — 0 lokale Dateien):

Der Cloud-Bronze-Consumer (`src/cloud_pubsub_consumer_bronze.py` via `run_pubsub_bronze.ps1`) zieht die Nachrichten per Pull-API aus `sub-youtube-ingest`, bestätigt sie (`acknowledge`), ergänzt UTC-Audit-Metadaten (`_ingested_at`, `_kafka_offset`, `_kafka_partition`), erzeugt Snappy-Parquet im Arbeitsspeicher (`io.BytesIO()`) und schreibt es **direkt in den GCS-Bucket** `gs://lakehouse-youtube-cloudanalytics-514018/bronze/`. Parallel isoliert `src/cloud_pubsub_consumer_dlq.py` (`run_pubsub_dlq.ps1`) alle fehlerhaften Nachrichten aus `sub-youtube-dlq` direkt nach `gs://lakehouse-youtube-cloudanalytics-514018/quarantine/`. Auf der lokalen Festplatte werden keinerlei Bronze- oder Quarantäne-Dateien abgelegt.

3. Silver Layer: Cloud-to-Cloud Transformation & Apache Iceberg V2 im GCS Bucket:

Das Modul `src/cloud_silver_gcs.py` (`run_silver.ps1` — bzw. der ephemere PySpark-Job `src/transform_silver.py`) liest die Bronze-Parquet-Dateien direkt aus `gs://lakehouse-youtube-cloudanalytics-514018/bronze/`, konvertiert Datentypen (`views`, `likes`, `dislikes`, `comment_count` nach `INT64`; `trending_date` von `yy.dd.MM` nach `DATE`; `publish_time` nach `TIMESTAMP`) und führt den Kategorie-Join mit `data/raw/US_category_id.json` aus. Das Ergebnis wird **ausschließlich in den Google Cloud Storage Bucket** geschrieben: sowohl als `gs://lakehouse-youtube-cloudanalytics-514018/silver/silver_videos.parquet` als auch in das **Apache Iceberg V2 Warehouse** `gs://lakehouse-youtube-cloudanalytics-514018/iceberg_warehouse/silver/videos/` (Metadaten & Daten). Lokale Ordner (`data/bronze`, `quarantine`, `silver`, `iceberg_warehouse`, `gold`) existieren nicht mehr.

4. Serverless End-to-End Orchestrierung via Google Cloud Workflows (GCS -> BigQuery -> Star-Schema -> Quality Gate):

Der per Terraform bereitgestellte Google Cloud Workflow `youtube-dataops-pipeline` (`infra/workflows/pipeline.yaml`, gestartet via `run_workflow.ps1` oder `run_pipeline.ps1`) steuert den **gesamten Cloud-DWH-Prozess End-to-End in 8 Schritten**: 1. `LOAD DATA OVERWRITE` direkt aus `gs://lakehouse-youtube-cloudanalytics-514018/silver/*.parquet` in die BigQuery-Tabelle `silver_videos`, 2. Row-Count-Check, 3. Aufbau des deduplizierten Staging-Views `stg_videos`, 4. Aufbau `dim_category`, 5. Aufbau `dim_date`, 6. Aufbau der SCD-2-Dimension `dim_video` (mit `LEAD()`-Window-Timeline über alle 3 Titelversionen von `2kyS6SvSYSE`), 7. Aufbau der Faktentabelle `fact_video_metrics` (Point-in-Time Join) und 8. Foreign-Key Quality Gate (`0 quality errors detected`).

5. Deklarative dbt Core Modelle, 15 Data Tests & Cloud Observability:

Komplementär zum Cloud Workflow stehen alle Modelle in **dbt Core** (`dbt/`) bereit und werden durch **15 automatisierte dbt-Tests** (`run_dbt.ps1 test` -> `PASS=15`) auf Eindeutigkeit, Not-Null und referenzielle Integrität geprüft. Sämtliche Laufzeit-, Kosten- und Streaming-Metriken werden ohne lokale Daemons direkt in **Grafana Cloud** (`dashboards/youtube_dataops_monitoring.json`) visualisiert.

3.2 Vollständige Repository-Struktur mit Versions-Zuordnung

```
DE_Capstone_2026/
|-- .github/workflows/                    # [v2.1 Active] CI/CD Automation (GitHub Actions)
|   |-- ci.yml                            # [v2.1 Active] Syntax-, Flake8-, YAML- & JSON-Validierung
|   +-- release.yml                       # [v2.1 Active] Automatische GitHub Releases bei Tag-Push (v*)
|-- dashboards/                           # [v2.1 Active] Observability-as-Code
|   +-- youtube_dataops_monitoring.json   # [v2.1 Active] Grafana Cloud Dashboard (Plugin: stackdriver)
|-- docs/                                 # [v2.1 Active] Enterprise Dokumentation & Handbuch
|   +-- YouTube_DataOps_Platform_2026_Architektur_Repertorium.pdf
|-- dags/                                 # [v1.0 Legacy] Lokale Airflow DAGs (abgeloest durch Workflows)
|-- dbt/                                  # [v2.0 Active] Cloud DWH Transformationen (Kimball & SCD-2)
|   |-- dbt_project.yml & profiles.yml    # [v2.0 Active] dbt Konfiguration & BigQuery EU-Profil
|   |-- snapshots/dim_video_snapshot.sql  # [v2.0 Active] SCD-2 Snapshot (check_cols: title, tags, ...)
|   +-- models/
|       |-- staging/stg_videos.sql        # [v2.0 Active] Idempotente Deduplizierung & Typisierung View
|       +-- marts/core/                   # [v2.0 Active] Kimball Star-Schema Marts & 15 Tests
|           |-- dim_video.sql             # [v2.0 Active] SCD-2 Dimension mit valid_from/to & is_current
|           |-- dim_category.sql          # [v2.0 Active] Conformed Kategorie-Dimension
|           |-- dim_date.sql              # [v2.0 Active] Kalender-Dimension (is_weekend, day_name)
|           |-- fact_video_metrics.sql    # [v2.0 Active] Fakten mit Point-in-Time SCD-2 Join
|           +-- schema.yml                # [v2.0 Active] 15 automatisierte dbt Quality Gate Tests
|-- infra/                                # [v2.1 Active] Infrastructure as Code (Terraform 1.17)
|   |-- versions.tf, variables.tf, main.tf# [v2.1 Active] 10 GCP Ressourcen (GCS, BQ, 2x Topic, 2x Sub, 3x
SP, Workflow)
|   |-- sql/01_sp_build_stg_videos.sql    # [v2.1 Active] Pattern B Stored Procedure: Idempotenter Staging
View
|   |-- sql/02_sp_build_dimensions.sql    # [v2.1 Active] Pattern B Stored Procedure: dim_category, dim_dat
e & SCD-2 dim_video
|   |-- sql/03_sp_refresh_fact_table.sql  # [v2.1 Active] Pattern B Stored Procedure: fact_video_metrics (P
oint-in-Time Join)
|   +-- workflows/pipeline.yaml           # [v2.1 Active] End-to-End Cloud Workflow (GCS Load -> CALL SPs -
> Quality Gate)
|-- data/                                 # [v1.0-v2.1] Lokale Ausgangslage (0 lokale Parquets/Icebergs!)
|   |-- raw/US_category_id.json           # [Ausgangslage] Statische YouTube-Kategorie-Referenz
|   +-- sample/us_videos_mini_500.csv     # [Ausgangslage] 500 YouTube-Trending-Events fuer das Streaming
|-- docker/                               # Modulare Docker Compose Definitionen
|   |-- docker-compose-spark.yml          # [v2.0 Active] Ephemerer PySpark + Iceberg V2 Container
|   |-- docker-compose-kafka.yml          # [v1.0/v2.0 Legacy] Lokaler Kafka KRaft (in v2.1 durch Pub/Sub e
rsetzt)
|   |-- docker-compose-airflow.yml        # [v1.0 Legacy] Alter Airflow 2.9 & Postgres Stack
|   +-- docker-compose-monitoring.yml     # [v1.0 Legacy] Alter lokaler Prometheus/Grafana Stack
|-- src/                                  # Python Quellcode
|   |-- cloud_pubsub_producer.py          # [v2.1 Active] Serverless GCP Pub/Sub Producer (10% DLQ + SCD-2)
|   |-- cloud_pubsub_consumer_bronze.py   # [v2.1 Active] Pull-Consumer -> Direkt in GCS (gs://.../bronze/)
|   |-- cloud_pubsub_consumer_dlq.py      # [v2.1 Active] Pull-Consumer -> Direkt in GCS (gs://.../quaranti
ne/)
|   |-- cloud_silver_gcs.py               # [v2.1 Active] Cloud-to-Cloud Silver & Iceberg V2 in GCS (0 loka
le Dateien)
|   |-- producer.py                       # [v1.0/v2.0 Legacy] Lokaler Kafka Producer (optional fuer Offlin
e-Tests)
|   |-- consumer_bronze.py & consumer_dlq.py # [v1.0/v2.0 Legacy] Lokale Kafka Consumer
|   |-- transform_silver.py               # [v2.0 Active] PySpark Broadcast-Join & Apache Iceberg V2
|   |-- transform_gold.py                 # [v1.0 Legacy] Altes Spark-Gold (abgeloest durch dbt)
|   |-- cloud_loader_bigquery.py          # [v2.0/v2.1 Active] Cloud-to-Cloud GCS -> BigQuery Loader
|   |-- cloud_workflow_runner.py          # [v2.1 Active] REST-Trigger & Live-Poller fuer den 8-Stufen Clou
d Workflow
|   +-- generate_repertorium_pdf.py       # [v2.1 Active] ReportLab Generator fuer das PDF-Repertorium
+-- run_pipeline.ps1 & *.ps1 Runner       # [v2.1 Active] One-Click Master-Runner & Einzel-Runner fuer Powe
rShell
```

4. Streaming Ingestion & Resilienz (Cloud Pub/Sub v2.1 vs. Kafka KRaft, DLQ & SCD-2)

In produktiven DataOps-Architekturen ist unkontrollierter Datenverlust inakzeptabel. Gleichzeitig darf ein korrupter Datensatz niemals die gesamte Ingestion-Pipeline blockieren ('Poison Pill Problem'). Dieses Kapitel erläutert den Evolutionssprung vom lokalen **Apache Kafka KRaft Broker** (`v1.0.0` / `v2.0.0`) hin zum vollständig serverlosen **Google Cloud Pub/Sub Streaming-Layer** in Release `v2.1.0`.

4.1 Warum ist Kafka in Release v2.1.0 optional? (Ablösung durch Google Cloud Pub/Sub)

In den Releases `v1.0.0` und `v2.0.0` lief lokal ein Docker-Container mit Apache Kafka im ZooKeeper-losen **KRaft-Modus** (`docker/docker-compose-kafka.yml`, `127.0.0.1:9092`). Obwohl KRaft gegenüber dem alten ZooKeeper-Setup aus 2021 bereits eine massive Verbesserung darstellte, blieb ein entscheidender Nachteil bestehen: Ein Kafka-Broker belegt dauerhaft lokalen Arbeitsspeicher und ist für Cloud-Dienste (wie Grafana Cloud) nicht nativ sichtbar.

In **Release** `v2.1.0` übernimmt deshalb **Google Cloud Pub/Sub** das gesamte Event-Streaming serverlos in der Cloud:

- **Keine lokale Broker-Infrastruktur nötig:** `start_kafka.ps1` muss in `v2.1.0` nicht mehr gestartet werden. Alle Nachrichten fließen verschlüsselt per HTTPS/REST direkt an `pubsub.googleapis.com`.
- **Deklarative Bereitstellung per Terraform:** `infra/main.tf` provisioniert automatisch die beiden Topics `topic-youtube-ingest` und `topic-youtube-dlq` sowie die beiden Pull-Subscriptions `sub-youtube-ingest` und `sub-youtube-dlq` (7 Tage Retention, 60s Ack-Deadline).
- **Native Cloud-Observability in Grafana Cloud:** Jeder Publish-Aufruf erzeugt automatisch die GCP-Systemmetrik `pubsub.googleapis.com/topic/send_message_operation_count`, die in Panel 3 und Panel 7 des Grafana-Cloud-Dashboards live nach `topic-youtube-ingest` vs. `topic-youtube-dlq` aufgeschlüsselt wird.

4.2 Die Resilienz-Architektur: 10% Fault-Injection, DLQ & Deterministische SCD-2 Mutation

Der serverlose Producer (`src/cloud_pubsub_producer.py` via `run_pubsub_producer.ps1`) liest alle **500 Datensätze** aus `data/sample/us_videos_mini_500.csv`, kodiert jedes JSON-Payload in Base64 und sendet es in gebündelten REST-Batches an Google Cloud Pub/Sub:

- **Sauberer Hauptstrom (`topic-youtube-ingest` -> `sub-youtube-ingest`):** Im verifizierten Live-Lauf wurden **444 valide Events** publiziert, von `src/cloud_pubsub_consumer_bronze.py` (`run_pubsub_bronze.ps1`) per Pull-API abgerufen, per `acknowledge` bestätigt und als `data/bronze/bronze_videos_*.parquet` (90,0 KB) gespeichert.
- **10% Fault-Injection & Dead Letter Queue (`topic-youtube-dlq` -> `sub-youtube-dlq`):** Mit `ERROR_RATE = 0.10` wurden **56 korrupte Events** (`views = 'CORRUPT_INVALID_NUMBER'`) in das Cloud-DLQ-Topic isoliert und von `src/cloud_pubsub_consumer_dlq.py` (`run_pubsub_dlq.ps1`) nach `data/quarantine/quarantine_videos_*.parquet` (26,3 KB) geschrieben.
- **Deterministische SCD-2 Mutation (`2kyS6SvSYSE`):** Das Zielvideo `2kyS6SvSYSE` ist von der Fault-Injection ausgenommen und durchläuft über drei Trending-Tage deterministisch drei Titel- und Tag-Mutationen (`Original` -> `(Official Update)` -> `[THE TRUTH]`).

Original-Codeausschnitt aus `src/cloud_pubsub_producer.py` (v2.1 Serverless Streaming):

```
# src/cloud_pubsub_producer.py (Original-Auszug: Serverless GCP Pub/Sub Producer)
PROJECT_ID = "cloudanalytics-514018"
TOPIC_INGEST = "topic-youtube-ingest"
TOPIC_DLQ = "topic-youtube-dlq"
SCD2_TARGET_VIDEO_ID = "2kyS6SvSYSE"

for row in reader:
    vid = row.get("video_id", "")
    if vid == SCD2_TARGET_VIDEO_ID:
        scd2_occurrence_count += 1
        if scd2_occurrence_count in SCD2_MUTATIONS:
            row.update(SCD2_MUTATIONS[scd2_occurrence_count])

    if vid != SCD2_TARGET_VIDEO_ID and random.random() < ERROR_RATE:
        corrupted_row = dict(row)
        corrupted_row["views"] = "CORRUPT_INVALID_NUMBER"
        corrupted_row["_error_reason"] = "Simulated fault injection: non-numeric views"
        dlq_batch.append(corrupted_row)
    else:
        clean_batch.append(row)

# Publiziert Base64-kodierte Nachrichten per HTTPS POST an pubsub.googleapis.com
publish_batch(session, TOPIC_INGEST, clean_batch)
publish_batch(session, TOPIC_DLQ, dlq_batch)
```

5. Ephemerer Compute & Open Table Format (Apache Iceberg V2)

Im Zeitalter moderner Cloud Lakehouses reichen lose Parquet-Dateien im Object Store für produktive Workloads nicht mehr aus. Unternehmen benötigen ACID-Transaktionen, Snapshot-Isolation, Schema-Evolution und Time-Travel. Dieses Kapitel beschreibt die Implementierung des Silver-Layers mit PySpark 3.5 und **Apache Iceberg V2** (`src/transform_silver.py`).

5.1 Warum lose Parquet-Dateien in Produktion scheitern

Parquet ist ein exzellentes spaltenorientiertes Dateiformat — aber ein Ordner voller Parquet-Dateien ist **keine Datenbanktabelle**:

- **Fehlende Atomarität (Dirty Reads):** Bricht ein Spark-Schreibjob nach 40 von 50 Parquet-Dateien ab, bleibt ein halb geschriebenes, korruptes Verzeichnis zurück.
- **Teures Directory Listing `O(N)`:** Abfrage-Engines müssen bei losen Dateien langsame Dateisystem-Listings (`listStatus`) über alle Unterordner ausführen.
- **Kein Time-Travel & kein sicheres Schema-Update:** Überschriebene Parquet-Dateien sind unwiderruflich verloren; Spaltenumbenennungen brechen bestehende Leser.

5.2 Die Apache Iceberg V2 Metadaten-Hierarchie (`local.silver.videos`)

Metadaten-Ebene	Physischer Pfad im Projekt	Funktion & ACID-Mechanismus
Iceberg Catalog	`spark.sql.catalog.local` (`type = hadoop`)	Verwaltet den atomaren Zeiger auf die aktuelle `vN.metadata.json` im Warehouse `/app/data/iceberg_warehouse`.
Table Metadata	`silver/videos/metadata/v1.metadata.json`	Speichert das Tabellen-Schema (Field-IDs), Partitionsspezifikationen und die vollständige Snapshot-Historie.
Manifest List	`silver/videos/metadata/snap-*.avro`	Verzeichnet alle Manifest-Dateien eines einzelnen Commits inkl. Partitions-Statistiken für schnelles Pruning.
Manifest Files	`silver/videos/metadata/*-m0.avro`	Trackt jede einzelne Parquet-Datendatei mit Zeilenanzahl und Spalten-Min/Max-Werten (Data Skipping).
Data Files	`silver/videos/data/*.parquet`	Die Snappy-komprimierten, typisierten und per Broadcast-Join angereicherten YouTube-Events.

5.3 Ephemerer Spark Compute, Broadcast-Join & Dual-Write

Über `run_silver.ps1` wird der Container `de_spark` (`docker/docker-compose-spark.yml`) on-demand gestartet, führt `src/transform_silver.py` aus und beendet sich danach sofort wieder (0 € Idle-Ressourcen). Dabei liest Spark die Kategorie-Referenz `data/raw/US_category_id.json` und erzwingt über `broadcast(df_categories)` einen **Zero-Shuffle Broadcast-Join** im Arbeitsspeicher. Anschließend schreibt der Job sowohl atomar in die Iceberg-Tabelle `local.silver.videos` als auch nach `/app/data/silver`, damit `src/cloud_loader_bigquery.py` die Daten nahtlos nach BigQuery hochladen kann:

```
# src/transform_silver.py (Original-Auszug: Iceberg Session, Broadcast Join & Commit)
def create_spark_session():
    return (
        SparkSession.builder.appName("DE_Capstone_Silver_Iceberg").master("local[*]")
        .config("spark.sql.extensions", "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions"
)
        .config("spark.sql.catalog.local", "org.apache.iceberg.spark.SparkCatalog")
```

```
        .config("spark.sql.catalog.local.type", "hadoop")
        .config("spark.sql.catalog.local.warehouse", "/app/data/iceberg_warehouse")
        .getOrCreate()
    )

# Typisierung & Datums-Harmonisierung ('17.14.11' -> yy.dd.MM)
df_cleaned = (
    df_bronze.withColumn("category_id", col("category_id").cast("string"))
    .withColumn("views", col("views").cast("long"))
    .withColumn("likes", col("likes").cast("long"))
    .withColumn("dislikes", col("dislikes").cast("long"))
    .withColumn("comment_count", col("comment_count").cast("long"))
    .withColumn("publish_time", to_timestamp(col("publish_time")))
    .withColumn("trending_date", to_date(col("trending_date"), "yy.dd.MM"))
)

# Zero-Shuffle Broadcast Join & atomarer Apache Iceberg V2 Commit
df_silver = df_cleaned.join(broadcast(df_categories), on="category_id", how="left").na.fill({"category_name
": "Unknown"})
df_silver.writeTo("local.silver.videos").createOrReplace()
spark.read.table("local.silver.videos.history").show(truncate=False)
```

6. Cloud Data Warehouse & Kimball Star-Schema (dbt Core & SCD-2)

Nachdem `src/cloud_loader_bigquery.py` (`load_to_bigquery.ps1`) die 295 bereinigten Silver-Events per `WRITE_TRUNCATE` als Quelltabelle `silver_videos` in das BigQuery-Dataset `youtube_analytics` (Location EU) geladen hat, übernimmt **dbt Core 1.13** (`run_dbt.ps1`) die In-Warehouse-Transformation (ELT) in ein dimensionales **Star-Schema nach Ralph Kimball**.

6.1 Die 6 dbt-Artefakte im BigQuery Dataset `youtube_analytics`

Modell / Tabelle	Materialisierung	Primär-/Surrogate-Key	Pfad & Transformations-Logik
stg_videos	View	`video_id` + `trending_date`	`models/staging/stg_videos.sql`: Trimming von Strings, Casting auf `INT64`, `DATE` und `TIMESTAMP`, Filter `video_id is not null`.
dim_video_snapshot	SCD-2 Snapshot Table	`dbt_scd_id` (MD5)	`snapshots/dim_video_snapshot.sql`: Überwacht `['title', 'channel_title', 'category_id', 'tags']` via `strategy='check'`.
dim_video	Table (Dimension)	`video_key` (`dbt_scd_id`)	`models/marts/core/dim_video.sql`: Exponiert `video_key`, `valid_from`, `valid_to` und berechnet das Flag `is_current`.
dim_category	Table (Conformed Dim)	`category_key` (`INT64`)	`models/marts/core/dim_category.sql`: Deduplizierte Kategorie-Dimension (`category_key`, `category_id`, `category_name`).
dim_date	Table (Date Spine)	`date_key` (`YYYYMMDD`)	`models/marts/core/dim_date.sql`: Berechnet `year`, `month`, `month_name`, `day`, `day_of_week`, `day_name` und `is_weekend`.
fact_video_metrics	Table (Fact Table)	`metric_key` (MD5 Hex)	`models/marts/core/fact_video_metrics.sql`: Tägliche Kennzahlen, `interaction_ratio` und Point-in-Time Join auf `dim_video`.

6.2 Deep Dive: SCD-2 Snapshot (`dim_video_snapshot.sql`) & Deduplizierung

Da ein Video im Streaming-Batch mehrfach vorkommen kann, dedupliziert der Snapshot die Quelle mittels `qualify row_number() over (partition by video_id order by trending_date desc, _ingested_at desc) = 1` und prüft auf Änderungen in den überwachten Attributen:

```
-- dbt/snapshots/dim_video_snapshot.sql (Original-Code)
{% snapshot dim_video_snapshot %}
{{
    config(
      target_schema='youtube_analytics',
      strategy='check',
      unique_key='video_id',
      check_cols=['title', 'channel_title', 'category_id', 'tags'],
      invalidate_hard_deletes=True
    )
}}
select video_id, title, channel_title, category_id, tags
from {{ ref('stg_videos') }}
qualify row_number() over (partition by video_id order by trending_date desc, _ingested_at desc) = 1
{% endsnapshot %}
```

6.3 Point-in-Time Join in der Faktentabelle (`fact_video_metrics.sql`)

Beim Aufbau der Faktentabelle wird jedes Trending-Event exakt mit derjenigen Version von `dim_video` verknüpft, die zum jeweiligen `trending_date` gültig war (`valid_from` bis `valid_to`). Zudem verhindert `safe_divide` Division-by-Zero-Fehler bei der Berechnung der `interaction_ratio`:

```
-- dbt/models/marts/core/fact_video_metrics.sql (Original-Code)
{{ config(materialized='table') }}

with source_data as ( select * from {{ ref('stg_videos') }} ),
dim_v as ( select * from {{ ref('dim_video') }} )

select
    to_hex(md5(concat(s.video_id, '_', cast(s.trending_date as string)))) as metric_key,
    dim_v.video_key,
    cast(s.category_id as int64) as category_key,
    cast(format_date('%Y%m%d', s.trending_date) as int64) as date_key,
    s.views, s.likes, s.dislikes, s.comment_count,
    safe_divide(s.likes + s.dislikes + s.comment_count, s.views) as interaction_ratio,
    s._ingested_at
from source_data s
inner join dim_v
    on s.video_id = dim_v.video_id
    and s.trending_date >= cast(dim_v.valid_from as date)
    and (dim_v.valid_to is null or s.trending_date < cast(dim_v.valid_to as date))
```

7. DataOps Quality Gates: 15 dbt Tests & In-Workflow Validierung

Gemäß dem DataOps-Prinzip **'Shift Left'** verlagert die Plattform Qualitätsprüfungen direkt in den Transformations- und Orchestrierungszyklus. Da Cloud Data Warehouses wie BigQuery standardmäßig keine Primary-Key- oder Foreign-Key-Constraints beim Schreiben erzwingen, implementiert das Projekt ein zweistufiges Schutzschild aus **15 deklarativen dbt Data Tests** (`dbt/models/marts/core/schema.yml`) und einem **serverlosen In-Workflow Quality Gate** (`infra/workflows/pipeline.yaml`).

7.1 Exakte Aufschlüsselung aller 15 dbt Data Tests (`schema.yml`)

Mart-Tabelle	Spalte	Definierte dbt-Tests	Anzahl	Abgesicherte Invariante
dim_video	`video_key` `video_id`	`unique`, `not_null` `not_null`	**3 Tests**	Jede SCD-2-Versionszeile hat einen eindeutigen Surrogate-Hash (`dbt_scd_id`) und einen Natural Key.
dim_category	`category_key`	`unique`, `not_null`	**2 Tests**	Keine doppelten oder leeren Kategorienschlüssel in der Conformed Dimension.
dim_date	`date_key`	`unique`, `not_null`	**2 Tests**	Jeder Kalendertag im Format `YYYYMMDD` existiert exakt einmal.
fact_video_metrics	`metric_key`	`unique`, `not_null`	**2 Tests**	Verhindert doppelte Faktenzeilen für dasselbe Video am selben Trending-Tag.
fact_video_metrics	`video_key`	`not_null` + `relationships` -> `dim_video`	**2 Tests**	**Referenzielle Integrität (SCD-2):** Jeder Fakt zeigt auf eine existierende `dim_video`-Version.
fact_video_metrics	`category_key`	`not_null` + `relationships` -> `dim_category`	**2 Tests**	**Referenzielle Integrität:** Jede Faktenzeile besitzt eine valide Kategorie.
fact_video_metrics	`date_key`	`not_null` + `relationships` -> `dim_date`	**2 Tests**	**Referenzielle Integrität:** Jede Faktenzeile besitzt einen gültigen Kalendertag.
SUMME	**Alle 4 Core Marts**	**4x unique, 8x not_null, 3x FK**	**15 Tests**	**Ergebnis im Produktivlauf:** `PASS=15 WARN=0 ERROR=0 SKIP=0`

7.2 Das serverlose In-Workflow Quality Gate (`pipeline.yaml`)

Zusätzlich zu `run_dbt.ps1 test` führt der Google Cloud Workflow bei jedem Lauf in der Cloud automatisch eine Foreign-Key-Orphan-Prüfung in BigQuery aus. Werden verwaiste Schlüssel (`invalid_count > 0`) entdeckt, wirft der Workflow im Schritt `evaluate_quality_gate` sofort eine Exception (`raise`) und verhindert eine Freigabe fehlerhafter Daten.

Formeller Qualitätsnachweis:
Verifiziertes Testergebnis (`run_dbt.ps1 test` & `run_workflow.ps1`):
• dbt Core: `Done. PASS=15 WARN=0 ERROR=0 SKIP=0 TOTAL=15`
• Cloud Workflow: `{"message":"DataOps Pipeline completed successfully. 0 quality errors detected.","status":"SUCCESS"}`

8. Infrastructure as Code (Terraform 1.17 Cloud Provisioning — 10 GCP Ressourcen & Stored Procedures)

Manuelles Zusammenklicken von Cloud-Ressourcen ('ClickOps') führt unweigerlich zu Konfigurations-Drift und nicht reproduzierbaren Umgebungen. In Release `v2.1.0` wird die gesamte Google Cloud Infrastruktur inklusive der **3 BigQuery Stored Procedures (`infra/sql/*.sql`) **über** Terraform 1.17** (`infra/main.tf`, `infra/variables.tf`, `infra/versions.tf`) deklarativ verwaltet und über den PowerShell-Wrapper `run_infra.ps1` gesteuert.

8.1 Die 10 bereitgestellten Google Cloud Ressourcen im Detail

Terraform Ressource	Live GCP Name / ID	Konfiguration & FinOps-Schutz
`google_storage_bucket .lakehouse_storage`	`lakehouse-youtube-cloudanalytics-514018`	Location `EU`, `uniform_bucket_level_access = true`, `versioning { enabled = true }`, `force_destroy = true`.
`google_bigquery_dataset .youtube_analytics`	`youtube_analytics`	Location `EU`, `delete_contents_on_destroy = true` (garantiert rückstandsloses Löschen bei `destroy`).
`google_pubsub_topic` **(2x)** `.youtube_ingest` / `.youtube_dlq`	`topic-youtube-ingest` `topic-youtube-dlq`	Serverlose Cloud-Pub/Sub-Topics für den regulären Event-Strom (~440 Clean) und die DLQ (~60 Fehler).
`google_pubsub_subscription` **(2x)** `.youtube_ingest_sub` / `.dlq_sub`	`sub-youtube-ingest` `sub-youtube-dlq`	Pull-Subscriptions für `run_pubsub_bronze.ps1` & `run_pubsub_dlq.ps1` (`ack_deadline_seconds = 20`).
`google_bigquery_routine` **(3x)** `.sp_build_stg_videos` `.sp_build_dimensions` `.sp_refresh_fact_table`	`sp_build_stg_videos` `sp_build_dimensions` `sp_refresh_fact_table`	**Pattern B (Stored Procedures):** Kapselt die formatierten SQL-Dateien aus `infra/sql/*.sql` als wartbare, in BigQuery direkt debuggbare Routinen.
`google_workflows_workflow .youtube_dataops_pipeline`	`youtube-dataops-pipeline`	Region `europe-west3` (Frankfurt), nutzt `var.service_account_email` und führt `workflows/pipeline.yaml` aus.

8.2 Auszug aus `infra/main.tf` (10 GCP Ressourcen inkl. Stored Procedures)

```
# infra/main.tf (Original-Auszug: Stored Procedures & Cloud Workflow)
resource "google_bigquery_routine" "sp_build_stg_videos" {
  dataset_id      = google_bigquery_dataset.youtube_analytics.dataset_id
  routine_id      = "sp_build_stg_videos"
  routine_type    = "PROCEDURE"
  language        = "SQL"
  definition_body = file("${path.module}/sql/01_sp_build_stg_videos.sql")
}

resource "google_bigquery_routine" "sp_build_dimensions" {
  dataset_id      = google_bigquery_dataset.youtube_analytics.dataset_id
  routine_id      = "sp_build_dimensions"
  routine_type    = "PROCEDURE"
  language        = "SQL"
  definition_body = file("${path.module}/sql/02_sp_build_dimensions.sql")
}

resource "google_bigquery_routine" "sp_refresh_fact_table" {
  dataset_id      = google_bigquery_dataset.youtube_analytics.dataset_id
  routine_id      = "sp_refresh_fact_table"
  routine_type    = "PROCEDURE"
  language        = "SQL"
  definition_body = file("${path.module}/sql/03_sp_refresh_fact_table.sql")
}

resource "google_workflows_workflow" "youtube_dataops_pipeline" {
  name            = "youtube-dataops-pipeline"
  region          = var.region
  service_account = var.service_account_email
  source_contents = file("${path.module}/workflows/pipeline.yaml")
  depends_on      = [
    google_bigquery_routine.sp_build_stg_videos,
    google_bigquery_routine.sp_build_dimensions,
    google_bigquery_routine.sp_refresh_fact_table
  ]
}
```

9. Serverlose Orchestrierung (Google Cloud Workflows & Pattern B Stored Procedures)

In Release `v2.1.0` löst **Google Cloud Workflows** den lokalen Airflow-Container-Stack aus `v1.0.0` vollständig ab. Während ein gemanagter Airflow-Cluster (Cloud Composer) durch dauerhaft laufende GKE-Knoten rund 300 € bis 450 € pro Monat kostet, arbeitet Cloud Workflows zu 100% serverlos mit **0,00 € Idle-Kosten** (5.000 Schritte/Monat im Always Free Tier inklusive).

9.1 Architektur-Pattern B: Trennung von Orchestrierung (`pipeline.yaml`) und SQL (`infra/sql/*.sql`)

Anstatt komplexe SQL-Statements als schwer wartbare Teilstrings in einer YAML-Datei zu zerstückeln, setzt die Plattform **Pattern B (BigQuery Stored Procedures + Cloud Workflows)** um:

- **Saubere `.sql`-Dateien mit vollem IDE-Support (`infra/sql/`):** Die SQL-Logik liegt formatiert und kommentiert in `01_sp_build_stg_videos.sql`, `02_sp_build_dimensions.sql` und `03_sp_refresh_fact_table.sql`. Dadurch lassen sich Queries jederzeit in der BigQuery-Konsole oder via `dbt` direkt debuggen und weiterentwickeln.
- **Schlanke `CALL`-Orchestrierung ohne 400-Zeichen-Limit:** `pipeline.yaml` lädt in Schritt 1 die Parquet-Daten per `LOAD DATA OVERWRITE` aus `gs://lakehouse-youtube-cloudanalytics-514018/silver/*.parquet` nach BigQuery, ruft danach nacheinander `CALL sp_build_stg_videos()`, `CALL sp_build_dimensions()` (inkl. SCD-2 `LEAD()`-Timeline für `2kyS6SvSYSE`) und `CALL sp_refresh_fact_table()` auf und sichert das Ergebnis über das Foreign-Key Quality Gate (`invalid_count == 0`) ab.

9.2 Vollständiger Code von `infra/workflows/pipeline.yaml` (Pattern B)

```
# infra/workflows/pipeline.yaml (Vollstaendiger Original-Code: Pattern B Stored Procedures)
main:
  params: [args]
  steps:
    - init_constants:
        assign:
          - project_id: "cloudanalytics-514018"
          - dataset_id: "youtube_analytics"
          - bucket_uri: "gs://lakehouse-youtube-cloudanalytics-514018/silver/*.parquet"
    - prepare_load_query:
        assign:
          - l_p1: ${"LOAD DATA OVERWRITE `" + project_id + "." + dataset_id + ".silver_videos` "}
          - l_p2: ${"FROM FILES (format = 'PARQUET', uris = ['" + bucket_uri + "'])"}
          - load_query: ${l_p1 + l_p2}
    - load_gcs_silver_to_bq:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${load_query}
            useLegacySql: false
        result: load_result
    - check_silver_data:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${"SELECT count(*) as total_records FROM `" + project_id + "." + dataset_id + ".silver_v
ideos`"}
            useLegacySql: false
        result: silver_check_result
    - build_stg_videos:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${"CALL `" + project_id + "." + dataset_id + ".sp_build_stg_videos`();"}
            useLegacySql: false
        result: stg_result
    - build_dimensions_scd2:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${"CALL `" + project_id + "." + dataset_id + ".sp_build_dimensions`();"}
            useLegacySql: false
        result: dims_result
    - refresh_fact_table:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${"CALL `" + project_id + "." + dataset_id + ".sp_refresh_fact_table`();"}
            useLegacySql: false
        result: fact_refresh_result
    - quality_gate_validation:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${"SELECT count(*) as invalid_keys FROM `" + project_id + "." + dataset_id + ".fact_vide
o_metrics` f LEFT JOIN `" + project_id + "." + dataset_id + ".dim_video` v ON f.video_key = v.video_key WHE
RE v.video_key IS NULL"}
            useLegacySql: false
        result: quality_result
    - get_quality_result:
```

```
        assign:
          - invalid_count: ${int(quality_result.rows[0].f[0].v)}
    - evaluate_quality_gate:
        switch:
          - condition: ${invalid_count > 0}
            raise: ${"Quality Gate FAILED! Found " + string(invalid_count) + " orphan records in fact_video
_metrics."}
    - finish_success:
        return:
          status: "SUCCESS"
          message: "Full End-to-End Cloud Lakehouse & Star-Schema Pipeline (Pattern B: Stored Procedures) c
ompleted successfully. 0 quality errors detected."
```

10. Cloud-Native Observability (Grafana Cloud & GCP Monitoring - Weg 1)

In Release `v2.1.0` wurde das Monitoring auf **'Weg 1: Cloud-Native Observability'** umgestellt. Anstatt lokale Prometheus- und Grafana-Container auf dem Entwickler-PC laufen zu lassen, verbindet sich **Grafana Cloud** über das offizielle **Google Cloud Monitoring** Plugin (`pluginId: stackdriver`) direkt mit den Telemetrie-APIs der Google Cloud Platform — mit **0 lokalen Daemons und 0 CPU/RAM-Belastung**.

10.1 Die 8 Panels des Grafana Dashboards (`youtube_dataops_monitoring.json`)

Row / Panel-Titel	Visualisierung	GCP Metric Type (`stackdriver`)	Operativer & FinOps Nutzen
1. Workflow Executions ***(Row: High-Level KPIs)***	Stat Panel (Grün)	`workflows.googleapis.com/finished_execution_count`	Gesamtzahl abgeschlossener Cloud-Workflow-Läufe (`REDUCE_SUM`, `ALIGN_DELTA`).
2. BigQuery Queries ***(Row: High-Level KPIs)***	Stat Panel (Blau)	`bigquery.googleapis.com/query/count`	Überwacht die DWH-Abfragefrequenz durch dbt und Cloud Workflows.
3. Pub/Sub Activity ***(Row: High-Level KPIs)***	Stat Panel (Violett)	`pubsub.googleapis.com/topic/send_message_operation_count`	Gesamtes Streaming-Nachrichtenvolumen über alle Cloud-Topics.
4. BigQuery Bytes Billed ***(Row: High-Level KPIs)***	Stat Panel (Ampel)	`bigquery.googleapis.com/query/scanned_bytes_billed`	**FinOps-Wächter:** Grün < 100 MB, Orange ab 100 MB, Rot ab 1 GB gescannten Daten.
5. Workflow Executions by Status ***(Row: Orchestration)***	Stacked Bar Chart	`workflows.googleapis.com/finished_execution_count`	Gruppiert nach `metric.label.status` (`SUCCEEDED` vs. `FAILED`) über die Zeit.
6. Workflow Execution Duration (p99) ***(Row: Orchestration)***	Time Series Line (s)	`workflows.googleapis.com/execution_durations`	Misst die 99.-Perzentil-Latenz (`ALIGN_PERCENTILE_99`) der Pipeline-Durchläufe.
7. Pub/Sub Topics (Ingest vs DLQ) ***(Row: DWH & DLQ Health)***	Stacked Bar Chart	`pubsub.googleapis.com/topic/send_message_operation_count`	Gruppiert nach `resource.label.topic_id` (`topic-youtube-ingest` vs. `topic-youtube-dlq`).
8. BigQuery Data Scanned Volume ***(Row: DWH & DLQ Health)***	Time Series Area	`bigquery.googleapis.com/query/scanned_bytes_billed`	Historischer Verlauf des Abfrage-Scanvolumens zur Erkennung teurer Full-Table-Scans.

Grafana Schema-Kompatibilität:
Technischer Hinweis zum Grafana-Import (`pluginId: stackdriver`): Intern identifiziert Grafana 10+ die Datenquelle 'Google Cloud Monitoring' unter der historischen Plugin-ID `stackdriver`. In `dashboards/youtube_dataops_monitoring.json` ist diese ID exakt hinterlegt, sodass beim JSON-Import sofort das Dropdown zur Auswahl der verbundenen GCP-Datenquelle erscheint.

11. DataOps CI/CD Pipelines, GitOps & Release-Management

Im Data Engineering trifft deklarativer Code auf persistenten Daten-State. Dieses Kapitel zeigt sowohl die im Repository aktiven **GitHub Actions Workflows** (`.github/workflows/ci.yml` und `release.yml`) als auch das **Enterprise-Zielbild für Multi-Environment Promotion (Dev -> Staging -> Prod)**.

11.1 Die aktiven GitHub Actions Workflows im Repository

```
# .github/workflows/ci.yml (Original-Code)
name: DataOps CI & Validation
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  validate-pipeline:
    name: Code Quality & Schema Verification
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Set up Python 3.10
        uses: actions/setup-python@v5
        with:
          python-version: "3.10"
          cache: "pip"
      - name: Install Python Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install flake8 pyyaml
      - name: Python Syntax & Lint Check
        run: |
          python -m compileall src/
          flake8 src/ --count --select=E9,F63,F7,F82 --show-source --statistics
      - name: Validate Cloud Workflows YAML
        run: |
          python -c "import yaml; yaml.safe_load(open('infra/workflows/pipeline.yaml'))"
      - name: Validate Grafana Dashboard JSON
        run: |
          python -c "import json; json.load(open('dashboards/youtube_dataops_monitoring.json'))"
```

Ergänzend triggert `.github/workflows/release.yml` bei jedem Push eines Versions-Tags (`v*`, wie `v1.0.0`, `v2.0.0`, `v2.1.0`) die Action `softprops/action-gh-release@v2` und publiziert automatisch ein GitHub Release inklusive generierter Release Notes.

11.2 Enterprise-Praxis: Drei-Zonen-Modell, Slim CI & Keyless OIDC

- **Umgebungs-Isolation (Dev / Staging / Prod):** Entwickler arbeiten lokal gegen ein isoliertes Dataset (`youtube_analytics_dev`). Pull Requests bauen in einer ephemeren Staging-Umgebung (`youtube_analytics_ci_<PR_ID>`). Auf `youtube_analytics_prod` hat kein Mensch Schreibrechte — ausschließlich der CI/CD-Service-Account.
- **dbt Slim CI (`state:modified+`):** In großen DWHs baut die CI-Pipeline bei einem Pull Request nicht alle 500 Modelle neu, sondern führt via `dbt build --select state:modified+ --defer --state ./prod-manifest` nur geänderte Modelle und deren direkte Kinder aus.
- **Keyless Authentication via Workload Identity Federation (OIDC):** Statt statischer JSON-Keys in GitHub Secrets tauscht GitHub Actions zur Laufzeit ein kurzlebiges OIDC-JWT-Token gegen ein 1 Stunde gültiges

GCP OAuth2-Token (`google-github-actions/auth@v2`).

- **Remote Terraform State Locking:** Der Terraform-State (`terraform.tfstate`) liegt im Enterprise-Betrieb in einem versionierten GCS-Bucket mit automatischem State-Locking, damit niemals zwei Pipelines gleichzeitig `terraform apply` ausführen.

12. Architectural Decision Records (ADRs 001–007) & Kritische Enterprise-Risikoanalyse

12.1 Die 7 zentralen Architekturentscheidungen (ADRs 001 bis 007)

ADR-001: Apache Kafka KRaft vs. ZooKeeper (v1.0/v2.0)

Entscheidung: Einsatz von Apache Kafka im ZooKeeper-losen KRaft-Modus (`docker-compose-kafka.yml`).
Kontext & Begründung: Das 2021er Setup litt unter ZooKeeper-Timeouts und Split-Brain-Problemen. KRaft halbiert die Container-Anzahl und verwaltet den Cluster-State in einem internen Raft-Quorum-Log.

ADR-002: Apache Iceberg V2 Table Format vs. Pure Parquet (ab v2.0)

Entscheidung: Nutzung von Apache Iceberg V2 (`local.silver.videos`) für den Silver Lakehouse Layer.
Kontext & Begründung: Reine Parquet-Ordner bieten keine ACID-Garantien und keine Snapshot-Isolation. Iceberg V2 trennt physische Parquet-Daten strikt von logischen Metadaten (`v1.metadata.json`, `snap-*.avro`), erlaubt atomare Commits, Time-Travel und Engine-Neutralität.

ADR-003: Ephemerer Docker PySpark vs. Permanenter Spark-Cluster

Entscheidung: Ausführung von PySpark-Jobs in einem ephemeren On-Demand-Container (`de_spark`).
Kontext & Begründung: Ein dauerhaft laufender Spark-Cluster verbraucht rund um die Uhr RAM/CPU. Ephemerer Compute erzeugt 0 € Leerlaufkosten und nutzt über `broadcast()` einen Zero-Shuffle Join.

ADR-004: Kimball Star-Schema & SCD-2 via dbt Core vs. One Big Table (ab v2.0)

Entscheidung: Modellierung als relationales Kimball Star-Schema mit SCD-2 Snapshot (`dim_video_snapshot`) in BigQuery.
Kontext & Begründung: Flache Tabellen (OBT) überschreiben bei Titeländerungen die Historie (SCD-1). Das Star-Schema trennt Fakten (`fact_video_metrics`) von Dimensionen (`dim_video`, `dim_category`, `dim_date`), ermöglicht Point-in-Time Joins und wird durch 15 dbt-Tests abgesichert.

ADR-005: Google Cloud Workflows vs. Apache Airflow / Cloud Composer (v2.1)

Entscheidung: Ablösung von Apache Airflow (v1.0) durch serverlose Google Cloud Workflows (v2.1).
Kontext & Begründung: Cloud Composer kostet mindestens ~300 €/Monat im Leerlauf. Cloud Workflows startet in unter 200 ms via REST, orchestriert BigQuery nativ, prüft das Foreign-Key Quality Gate in der Cloud und kostet im Always Free Tier 0,00 €.

ADR-006: Grafana Cloud Direct Monitoring (Weg 1) vs. Lokale Prometheus-Container (v2.1)

Entscheidung: Direkte Anbindung von Grafana Cloud an Google Cloud Monitoring (`pluginId: stackdriver`).
Kontext & Begründung: 0 lokale Daemons, direkte Abfrage von `workflows`, `bigquery` und `pubsub` Metriken über den Service Account (`roles/monitoring.viewer`) und versioniertes Dashboard-as-Code.

ADR-007: Serverless Google Cloud Pub/Sub (v2.1) vs. Lokaler Docker-Kafka-Broker (v1.0/v2.0)

Entscheidung: Ablösung des lokalen Kafka-Containers durch Google Cloud Pub/Sub (`topic-youtube-ingest`/`sub-youtube-ingest` & `topic-youtube-dlq`/`sub-youtube-dlq`) in Release `v2.1.0`.
Kontext & Begründung: Ein lokaler Kafka-Broker belegt dauerhaft RAM und ist für Cloud-Dienste nicht nativ sichtbar. Cloud Pub/Sub skaliert serverlos, benötigt 0 lokale Container, kostet bis 10 GB/Monat 0,00 € und liefert Echtzeit-Telemetrie an Grafana Cloud.

12.2 Kritische Analyse: 8 Reale Risiken & Architektonische Schwachstellen im Enterprise-Umfeld

Eine belastbare Senior/Lead-Architekturdokumentation verschweigt nicht die Grenzen des eigenen Designs. Während diese Plattform für ein 0-€-Kostenprofil (Always Free Tier) und deterministische Reproduzierbarkeit optimiert wurde, birgt sie beim Skalieren auf **Terabyte- bis Petabyte-Volumina im Enterprise-Betrieb** acht konkrete strukturelle Risiken und Schwachstellen, die wie folgt mitigiert werden müssen:

1. 100% GCS Cloud Storage erreicht — verbleibender lokaler Trigger-Prozess für Pub/Sub & Silver

Erreichte Härtung in `v2.1.0`: Abseits der Ausgangslage (`data/raw` & `data/sample`) liegen **0 Parquet- oder Iceberg-Dateien lokal**; Bronze, Quarantine, Silver und das Apache Iceberg V2 Warehouse liegen zu 100% im GCS-Bucket `gs://lakehouse-youtube-cloudanalytics-514018/`, und der Cloud Workflow lädt direkt per `LOAD DATA OVERWRITE` aus GCS nach BigQuery.
Verbleibendes Enterprise-Risiko & Mitigation: Das Anstoßen der Python-Skripte (`run_pipeline.ps1`) erfolgt im Capstone noch von der Workstation aus. Im 24/7-Enterprise-Betrieb werden die Consumer durch eine native **Pub/Sub Cloud Storage Subscription** (bzw. **Cloud Run Jobs**) ersetzt und vom Cloud Workflow per HTTP-Connector getriggert.

2. Lokaler Terraform-State & Statischer Service-Account-JSON-Schlüssel

Ist-Zustand (Schwachstelle): Die Datei `infra/terraform.tfstate` liegt lokal und unverschlüsselt auf dem Host; die Authentifizierung nutzt `gcp_service_account.json` (langlebiger privater Schlüssel mit Exfiltrationsrisiko).
Enterprise-Mitigation: Auslagerung des Terraform-States in einen versionierten **GCS Remote Backend Bucket** mit nativem State-Locking sowie Umstellung von statischen JSON-Keys auf **Workload Identity Federation (OIDC)** (kurzlebige 1h-OAuth2-Tokens).

3. Intra-Batch-Mutationsverlust bei `dbt snapshot` — Gelöst durch `LEAD()`-Window-Timeline in `dim_video`

Hintergrund (Schwachstelle reiner Snapshot-Tabellen): Ein einzelner `dbt snapshot`-Lauf kann pro `video_id` nur einen Zustand pro Batch vergleichen und würde Zwischenmutationen desselben Batches verlieren.
Implementierte Lösung in `v2.1.0`: Sowohl Schritt 6 des Cloud Workflows (`build_dim_video_scd2` in `pipeline.yaml`) als auch `dbt/models/marts/core/dim_video.sql` berechnen die SCD-2-Versionsgrenzen (`valid_from`, `valid_to`, `is_current`) deterministisch per `LEAD(CAST(v_start AS TIMESTAMP)) OVER (PARTITION BY video_id ORDER BY v_start ASC)`, sodass alle 3 historischen Titelversionen von `2kyS6SvSYSE` auch innerhalb eines einzigen Micro-Batches lückenlos erhalten bleiben.

4. Full-Refresh (`CREATE OR REPLACE TABLE`) statt Inkrementeller Partitionierung

Ist-Zustand (Schwachstelle): Sowohl `load_to_bigquery.py` (`WRITE_TRUNCATE`) als auch die dbt-Marts und der Cloud Workflow bauen `fact_video_metrics` bei jedem Lauf komplett neu auf. Bei 500 Zeilen dauert dies 2 Sekunden — bei **50 Terabyte** führt ein Full-Table-Scan zu enormen BigQuery-Kosten (~300 $ pro Lauf).
Enterprise-Mitigation: Umstellung von `fact_video_metrics` in dbt auf `materialized='incremental'`, `incremental_strategy='merge'`, `partition_by={'field': 'trending_date', 'data_type': 'date'}` und `cluster_by=['category_key', 'video_key']`.

5. Überprivilegierter Service Account (`*.admin` statt Least-Privilege Principle)

Ist-Zustand (Schwachstelle): Ein einziger Service Account vereint `bigquery.admin`, `storage.admin`, `pubsub.admin` und `workflows.admin` sowohl für das Terraform-Deployment als auch für die Laufzeit des Cloud Workflows.
Enterprise-Mitigation: Strikte Trennung (Segregation of Duties) in mindestens drei Service Accounts: 1. `sa-terraform-deployer` (nur in CI/CD), 2. `sa-pubsub-ingest` (nur `pubsub.publisher`/`subscriber`), 3. `sa-workflow-runtime` (nur `bigquery.dataEditor` und `bigquery.jobUser`).

6. Fehlende Schema-Durchsetzung am Ingest-Rand (Schemaless JSON vs. Schema Registry)

Ist-Zustand (Schwachstelle): Die Producer prüfen fehlerhafte `views` im Python-Code der Quelle und senden freies JSON. Wenn ein Drittsystem direkt fehlerhaftes JSON in `topic-youtube-ingest` publiziert, wird dies erst beim nachgelagerten Pull/Cast bemerkt.
Enterprise-Mitigation: Kopplung von `topic-youtube-ingest` an eine **Google Cloud Pub/Sub Schema Registry** (Avro oder Protocol Buffers) sowie Einsatz einer nativen Pub/Sub `dead_letter_policy` mit `max_delivery_attempts = 5` direkt auf der Subscription.

7. Post-Write Quality Gate in `pipeline.yaml` (Gefahr kurzzeitiger Dirty Reads)

Ist-Zustand (Schwachstelle): In `infra/workflows/pipeline.yaml` überschreibt Schritt 4 (`refresh_fact_table`) zuerst die produktive Tabelle `fact_video_metrics`, und erst Schritt 5 (`quality_gate_validation`) prüft auf verwaiste Fremdschlüssel. Schlägt das Gate fehl, stehen die fehlerhaften Daten bereits in der Tabelle.
Enterprise-Mitigation: Umsetzung des **Write-Audit-Publish (WAP)** Patterns: Der Workflow schreibt zuerst in eine isolierte Staging-Tabelle `fact_video_metrics_wap`, führt darauf das Quality Gate aus und kopiert sie erst bei `invalid_count == 0` atomar per `CREATE OR REPLACE TABLE ... CLONE` in die Produktion.

8. Rein statische CI-Pipeline ohne Live-Datenbank-Integrationstests (Slim CI)

Ist-Zustand (Schwachstelle): `.github/workflows/ci.yml` prüft Python-Syntax, Flake8-Regeln sowie YAML/JSON-Strukturen, führt aber kein `dbt build` gegen BigQuery aus, da keine Cloud-Credentials in GitHub hinterlegt sind. Ein subtiler SQL-Tippfehler würde erst beim Deployment auffallen.
Enterprise-Mitigation: Anbindung von GitHub Actions per OIDC an ein dediziertes GCP-Testprojekt und Ausführung von `dbt build --select state:modified+ --defer --state ./prod-manifest` (Slim CI) in einem ephemeren Pull-Request-Dataset `youtube_analytics_ci_<PR_ID>`.

13. Deppensicheres Betriebshandbuch (9-Schritte-Anleitung & Playbooks)

Dieses Kapitel enthält die vollständige, **deppensichere Schritt-für-Schritt-Anleitung (End-to-End)** für das aktuelle Produktions-Release `v2.1.0` inklusive aller Voraussetzungen, IAM-Rollen, GCP-APIs, PowerShell-Runner und Fehlerbehebungs-Playbooks.

13.1 Checkliste der Voraussetzungen (IAM-Rollen & GCP APIs)

- **Lokale Systemumgebung:** Windows 10/11 mit PowerShell, Docker Desktop (gestartet, WSL2-Backend für den ephemeren Silver-PySpark-Job), Python 3.10+ sowie Terraform 1.17.
- **GCP Projekt & Service Account Key:** Projekt-ID `cloudanalytics-514018` (Region `europe-west3` / Location EU). Der JSON-Key des Service Accounts `cloudanalytics-514018@cloudanalytics-514018.iam.gserviceaccount.com` liegt als `gcp_service_account.json` im Projekt-Root (via `.gitignore` geschützt).
- **Die 6 zwingend erforderlichen GCP IAM-Rollen auf dem Service Account:**
  1. `roles/bigquery.admin` (BigQuery Administrator)
  2. `roles/storage.admin` (Storage Administrator)
  3. `roles/pubsub.admin` (Pub/Sub Administrator — für Topics & Pull-Subscriptions)
  4. `roles/workflows.admin` (Workflows Administrator)
  5. `roles/iam.serviceAccountUser` (Dienstkonto-Nutzer — zwingend für Workflow-Deployment!)
  6. `roles/monitoring.viewer` (Monitoring-Betrachter — zwingend für Grafana Cloud!)
- **Die 5 zwingend aktivierten Google Cloud APIs:** `workflows.googleapis.com`, `monitoring.googleapis.com`, `pubsub.googleapis.com`, `bigquery.googleapis.com`, `storage.googleapis.com`.

13.2 Die End-to-End Ausführungssequenz für Release v2.1.0 (1-Klick oder Einzelbefehle)

```
# SCHRITT 1: Python Virtual Environment aktivieren & Abhaengigkeiten installieren
cd D:\Projekte\CloudEng\DE_Capstone_2026
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt

# SCHRITT 2: Cloud-Infrastruktur (7 GCP Ressourcen inkl. 8-Stufen Workflow) per Terraform ausrollen
.\run_infra.ps1 init
.\run_infra.ps1 plan
.\run_infra.ps1 apply     # -> Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

# OPTION A (EMPFOHLEN): Komplette Cloud-Pipeline mit einem einzigen Befehl durchlaufen lassen!
.\run_pipeline.ps1        # Fuehrt Schritt 3 bis 5 vollautomatisch End-to-End in der Cloud aus!

# OPTION B: Einzel-Ausfuehrung der Cloud-Schritte (100% GCS Storage — 0 lokale Parquets/Icebergs):
# SCHRITT 3: Serverless Streaming Ingestion via Cloud Pub/Sub direkt in den GCS Bucket
.\run_pubsub_producer.ps1 # Sendet ~440 Clean Events (topic-youtube-ingest) + ~60 DLQ Events (topic-youtube
-dlq)
.\run_pubsub_bronze.ps1   # Zieht Clean-Events -> direkt nach gs://lakehouse-youtube-.../bronze/*.parquet
.\run_pubsub_dlq.ps1      # Isoliert DLQ-Events -> direkt nach gs://lakehouse-youtube-.../quarantine/*.parq
uet

# SCHRITT 4: Cloud-to-Cloud Silver & Apache Iceberg V2 direkt im GCS Bucket
.\run_silver.ps1          # Schreibt gs://.../silver/silver_videos.parquet & gs://.../iceberg_warehouse/

# SCHRITT 5: 8-Stufen Google Cloud Workflow in Frankfurt (europe-west3) ausfuehren
.\run_workflow.ps1        # Steuert GCS->BQ Load, stg_videos, Dims (SCD-2), Fact & Quality Gate -> SUCCEEDE
D

# SCHRITT 6: 15 automatisierte dbt Data-Quality-Tests gegen BigQuery validieren
.\run_dbt.ps1 test        # -> PASS=15 WARN=0 ERROR=0 SKIP=0 TOTAL=15

# SCHRITT 7: Grafana Cloud verbinden (Google Cloud Monitoring -> gcp_service_account.json)
#            & Dashboard importieren: dashboards/youtube_dataops_monitoring.json

# SCHRITT 8: Aufraeumen & 0-EUR-Kostengarantie nach Testende
.\run_infra.ps1 destroy   # Loescht alle 7 GCP-Ressourcen rueckstandslos (0,00 EUR Kosten)
```

13.3 Vollständige Referenz aller PowerShell-Runner (`.ps1`)

Runner-Skript	Parameter	Release-Status & Funktion
`run_pipeline.ps1`	Keine	**[v2.1 Master-Runner]** 1-Klick End-to-End Lauf (Pub/Sub -> GCS -> Silver/Iceberg GCS -> 8-Stufen Cloud Workflow).
`run_infra.ps1`	`init`, `plan`, `apply`, `destroy`	**[v2.1 Active]** Steuert Terraform 1.17 in `infra/` (7 GCP-Ressourcen).
`run_pubsub_producer.ps1`	Keine	**[v2.1 Active]** Serverless GCP Pub/Sub Producer (~440 Clean, ~60 DLQ, SCD-2 Mutationen).
`run_pubsub_bronze.ps1`	Keine	**[v2.1 Active]** Pull-Consumer (`sub-youtube-ingest` -> direkt `gs://.../bronze/*.parquet`, 0 lokal).
`run_pubsub_dlq.ps1`	Keine	**[v2.1 Active]** Pull-Consumer (`sub-youtube-dlq` -> direkt `gs://.../quarantine/*.parquet`, 0 lokal).
`run_silver.ps1`	Keine	**[v2.0/v2.1 Active]** Cloud-to-Cloud Silver & Iceberg V2 direkt im GCS Bucket (`gs://.../silver/` & `iceberg_warehouse/`).
`run_workflow.ps1`	Keine	**[v2.1 Active]** Startet den 8-Stufen End-to-End Cloud Workflow (GCS->BQ->Star-Schema->Quality Gate).
`load_to_bigquery.ps1`	Keine	**[v2.0/v2.1 Active]** Cloud-to-Cloud GCS -> BigQuery Loader (in v2.1 bereits in Schritt 1 von `run_workflow.ps1` enthalten).
`run_dbt.ps1`	`debug`, `snapshot`, `run`, `test`	**[v2.0/v2.1 Active]** Steuert dbt Core CLI (insb. `run_dbt.ps1 test` für alle 15 Quality-Gate-Tests).
`start_kafka.ps1` / `stop_kafka.ps1`	Keine	*[v1.0/v2.0 Legacy / Optional]* Lokaler Docker-Kafka-Stack (in v2.1 durch Pub/Sub ersetzt).
`run_producer.ps1`, `run_bronze.ps1`, `run_dlq.ps1`	Keine	*[v1.0/v2.0 Legacy / Optional]* Lokale Kafka Producer/Consumer (in v2.1 durch `run_pubsub_*.ps1` ersetzt).
`run_gold.ps1`	Keine	*[v1.0 Legacy]* Altes Spark-Gold-Skript (ab v2.0 durch `run_dbt.ps1` abgelöst).
`start_monitoring.ps1` / `stop_monitoring.ps1`	Keine	*[v1.0 Legacy]* Lokaler Prometheus/Grafana Stack (in v2.1 durch Grafana Cloud abgelöst).
`start_airflow.ps1` / `stop_airflow.ps1`	Keine	*[v1.0 Legacy]* Lokaler Airflow 2.9 Stack (in v2.1 durch Cloud Workflows abgelöst).

13.4 Produktions-Troubleshooting Playbooks (Aus echten Fehlern gelernt)

Playbook 1: GCP IAM `Error 403: Permission denied` beim Terraform Workflow-Deployment

• *Symptom:* `terraform apply` schlägt beim Erstellen von `google_workflows_workflow` mit `403 Permission 'workflows.workflows.create' or 'iam.serviceAccounts.actAs' denied` fehl.
• *Ursache:* Zum Binden eines Service Accounts an einen Cloud Workflow benötigt der ausführende Account neben `roles/workflows.admin` zwingend auch `roles/iam.serviceAccountUser`.
• *Lösung:* Den existierenden Service Account (`var.service_account_email`) in `infra/main.tf` referenzieren und ihm in der GCP Console `roles/workflows.admin` sowie `roles/iam.serviceAccountUser` zuweisen.

Playbook 2: Cloud Workflows Parser-Fehler (`400 characters limit` & `multiple step types`)

• *Symptom 1:* `maximum length for an expression is 400 characters, the provided expression has a length of 743`.
• *Symptom 2:* `step 'evaluate_quality_gate': step matches multiple types: 'switch', 'assign'`.
• *Lösung:* In `infra/workflows/pipeline.yaml` lange SQL-Abfragen in mehrere Variablen (`q_target`, `q_select`, `q_from`, `q_join`) aufteilen und `assign` (`get_quality_result`) strikt vom nachfolgenden `switch`-Schritt (`evaluate_quality_gate`) trennen.

Playbook 3: Grafana Cloud Import meldet `No data sources of type cloudmonitoring found`

• *Symptom:* Beim Import des Dashboard-JSONs lässt sich die konfigurierte Google Cloud Monitoring Datenquelle nicht auswählen.
• *Ursache:* Grafana verwendet intern die Plugin-ID `stackdriver` (nicht `cloudmonitoring`).
• *Lösung:* In `dashboards/youtube_dataops_monitoring.json` ist `"pluginId": "stackdriver"` und `"type": "stackdriver"` gesetzt. Zudem benötigt der Service Account die Rolle `roles/monitoring.viewer`.

14. Data Engineering Interview Masterclass & Technical Defense (25 Deep-Dive Q&As)

Dieses Kapitel dient als umfassendes Repetitorium für technische Architektur-Reviews und Vorstellungsgespräche auf Senior- und Lead-Data-Engineer-Niveau. Es liefert präzise, praxiserprobte Antworten auf **25 tiefgehende Fachfragen** rund um dieses Projekt.

F1: Warum haben Sie sich für Apache Iceberg V2 anstelle von reinem Parquet entschieden?

Parquet ist lediglich ein Dateiformat, keine Tabellen-Abstraktion. Lose Parquet-Dateien bieten keine ACID-Transaktionen (Gefahr von Dirty Reads bei Jobabbrüchen), kein Time-Travel und erfordern teure Directory-Listings. Apache Iceberg V2 verwaltet Metadaten hierarchisch (`v1.metadata.json` -> `snap-*.avro` -> `*-m0.avro`), garantiert atomare Commits per Compare-and-Swap und ermöglicht Data Skipping über Spaltenstatistiken.

F2: Warum haben Sie Google Cloud Workflows statt Apache Airflow (Cloud Composer) gewählt?

Eine bewusste FinOps- und Architekturentscheidung: Cloud Composer benötigt einen dauerhaft laufenden GKE-Cluster (~300 €/Monat), selbst wenn die Pipeline nur wenige Minuten am Tag läuft. Da unsere Transformationslogik vollständig in BigQuery liegt (ELT), benötigen wir lediglich eine leichtgewichtige State-Machine zur Orchestrierung. Cloud Workflows startet in Millisekunden, ist 100% serverlos und kostet im Always Free Tier exakt 0,00 €.

F3: Wie handhaben Sie Slowly Changing Dimensions (SCD-2) und warum reicht SCD-1 nicht?

Bei SCD-1 überschreibt eine Titeländerung (wie bei Video `2kyS6SvSYSE`) den alten Wert — historische Analysen über den Effekt von Titel-Mutationen auf Klickraten sind damit unmöglich. Unser dbt Snapshot (`dim_video_snapshot`) implementiert SCD-2 (`strategy='check'`), schließt alte Versionen mit `valid_to` ab und legt eine neue Zeile mit `is_current = true` an. In `fact_video_metrics` ordnet ein Point-in-Time Join jedes Event exakt der damals gültigen Version zu.

F4: Was ist der Unterschied zwischen den 15 dbt-Tests und dem In-Workflow Quality Gate?

Die 15 dbt-Tests (`schema.yml`) prüfen beim Build-Prozess alle 4 Core-Marts auf `unique`, `not_null` und `relationships`. Das In-Workflow Quality Gate (`pipeline.yaml`) ist dagegen ein Laufzeit-Wächter direkt in der Cloud-Orchestrierung: Nach jedem Refresh von `fact_video_metrics` zählt ein `LEFT JOIN`-Check verwaiste Fremdschlüssel und wirft bei `invalid_count > 0` sofort eine Exception (`raise`).

F5: Wie verhindern Sie, dass fehlerhafte Streaming-Nachrichten die Pipeline zum Absturz bringen?

Durch das Dead-Letter-Queue-Muster (DLQ): `src/cloud_pubsub_producer.py` (bzw. `src/producer.py`) injiziert 10% korrupte Nachrichten (`views = 'CORRUPT_INVALID_NUMBER'`), leitet diese aber sofort in das Quarantäne-Topic `topic-youtube-dlq` um, während die 444 validen Events über `topic-youtube-ingest` ungestört verarbeitet werden. `src/cloud_pubsub_consumer_dlq.py` persistiert die 56 Fehler-Events samt `_error_reason` unter `data/quarantine/`.

F6: Warum wird das Zielvideo `2kyS6SvSYSE` im Producer von der 10% Fault-Injection ausgenommen?

Um 100% deterministische, reproduzierbare End-to-End-Tests für SCD-2 zu garantieren (`if vid != SCD2_TARGET_VIDEO_ID and random.random() < ERROR_RATE`). Würde eine der drei Titel-Mutationen zufällig in der DLQ landen, wäre die SCD-2-Versionskette nicht deterministisch testbar.

F7: Warum verbinden sich die lokalen Kafka-Skripte mit `127.0.0.1:9092` statt `localhost:9092`?

Unter Windows 10/11 löst `localhost` bevorzugt auf IPv6 (`::1`) auf. Wenn der Docker-Port-Forwarder auf IPv4 bindet, verursacht jeder Verbindungsaufbau einen 2-sekündigen TCP-SYN-Timeout vor dem Fallback auf IPv4. `127.0.0.1:9092` erzwingt sofortiges IPv4 ohne Disconnect-Lag.

F8: Warum führt `src/transform_silver.py` einen Dual-Write (Iceberg V2 + Parquet) durch?

Die Iceberg-Tabelle `local.silver.videos` liefert das volle Open-Table-Format mit Snapshot-Historie und ACID-Garantien im Lakehouse. Der zusätzliche Parquet-Export nach `/app/data/silver` dient als schlanker Transfer-Staging-Puffer für den lokalen Upload-Client `src/cloud_loader_bigquery.py` nach BigQuery.

F9: Warum ist der Broadcast-Join in PySpark so entscheidend für die Performance?

Ein Standard-SortMergeJoin müsste sowohl die großen Video-Events als auch die Kategorien über das Netzwerk nach `category_id` shuffeln. Da `US_category_id.json` winzig ist (< 50 KB), sendet `broadcast(df_categories)` die Lookup-Tabelle einmalig in den RAM jedes Executors. Der Join erfolgt rein lokal im Speicher (Zero Shuffle).

F10: Wie garantieren Sie Idempotenz über alle Pipeline-Stufen hinweg?

1. Silver Iceberg: `.writeTo('local.silver.videos').createOrReplace()` ersetzt den Snapshot atomar. 2. BigQuery Loader: `WriteDisposition.WRITE_TRUNCATE` überschreibt `silver_videos` ohne Duplikate. 3. dbt Staging (`stg_videos.sql`): Dedupliziert per `QUALIFY row_number() OVER (PARTITION BY video_id, trending_date ORDER BY _ingested_at DESC) = 1`. 4. dbt Marts & Cloud Workflow: Aktualisieren Tabellen atomar per `CREATE OR REPLACE TABLE` mit deterministischen MD5-Schlüsseln (`metric_key`).

F11: Warum haben Sie in `pipeline.yaml` das SQL-Statement in vier Variablen aufgeteilt?

Google Cloud Workflows besitzt ein hartes Parser-Limit von 400 Zeichen pro `${...}`-Ausdruck. Ein einzelner Ausdruck mit 743 Zeichen wird beim Deployment abgelehnt. Durch die Aufteilung in `q_target`, `q_select`, `q_from` und `q_join` bleibt jeder Teilstring unter 400 Zeichen und wird erst in `${q_target + q_select + q_from + q_join}` zusammengefügt.

F12: Wie optimieren Sie BigQuery-Kosten bei wachsendem Datenvolumen (Terabyte-Scale)?

Im On-Demand-Tarif berechnet BigQuery nach gescannten Bytes. Bei großen Datenmengen partitionieren wir `fact_video_metrics` nach `trending_date` (`PARTITION BY trending_date`) und clustern nach `category_key, video_key` (`CLUSTER BY`). Zusätzlich überwacht unser Grafana-Dashboard live die Metrik `bigquery.googleapis.com/query/scanned_bytes_billed`.

F13: Warum verwendet man Surrogate Keys (`video_key`) statt Natural Keys (`video_id`) im Star-Schema?

Weil `video_id` in einer SCD-2-Dimension (`dim_video`) nicht mehr eindeutig ist: Dasselbe Video hat bei 3 Titelversionen 3 Zeilen mit derselben `video_id`. Erst der Surrogate Key (`video_key = dbt_scd_id`) identifiziert exakt die historische Version, auf die die Faktentabelle verweist.

F14: Wie funktioniert CI/CD in einer echten Enterprise-Produktionsumgebung (GitOps)?

Kein Entwickler führt in Produktion manuell `dbt run` oder `terraform apply` aus. Änderungen laufen über Pull Requests: GitHub Actions authentifiziert sich keyless per OIDC (Workload Identity) gegen GCP, führt `terraform plan` und einen dbt Slim-CI-Lauf (`state:modified+`) in einem isolierten CI-Dataset aus. Erst nach Code-Review und Merge auf `main` rollt die CD-Pipeline die Änderungen automatisch nach `prod` aus.

F15: Welche Komponenten aus Release v1.0.0 und v2.0.0 sind in Release v2.1.0 obsolet und warum?

1. Lokales Docker-Kafka (`start_kafka.ps1`, `run_producer.ps1`, `run_bronze.ps1`, `run_dlq.ps1`) wurde in `v2.1.0` durch serverloses Google Cloud Pub/Sub (`run_pubsub_*.ps1`) abgelöst. 2. Lokaler Airflow (`start_airflow.ps1`, `dags/`) wurde durch Google Cloud Workflows ersetzt. 3. Lokaler Prometheus/Grafana (`start_monitoring.ps1`) wurde durch Grafana Cloud ('Weg 1' direkt auf GCP Monitoring) ersetzt. 4. Das alte Spark-Gold-Skript (`run_gold.ps1`) wurde bereits ab `v2.0.0` durch das dbt Kimball Star-Schema in BigQuery ersetzt.

F16: Worin unterscheiden sich Google Cloud Pub/Sub (v2.1) und Apache Kafka KRaft (v1.0/v2.0) architektonisch?

Kafka basiert auf einem partitionierten Commit-Log mit fortlaufenden Offsets pro Consumer-Group und erfordert laufende Broker-Instanzen; Ordering wird pro Partition garantiert. Cloud Pub/Sub ist dagegen ein global verteilter, serverloser Message-Bus mit Entkopplung von Topics und Subscriptions (`sub-youtube-ingest`), Message-Level Ack-Deadlines (60s) und 0,00 € Idle-Kosten ohne jegliche Cluster-Wartung.

F17: Was ist das Write-Audit-Publish (WAP) Pattern und warum ist es sicherer als ein Post-Write Quality Gate?

Bei unserem aktuellen Post-Write Gate in `pipeline.yaml` wird erst `fact_video_metrics` überschrieben und danach auf verwaiste Fremdschlüssel geprüft. Schlägt der Check fehl, sehen BI-Nutzer bereits korrupte Daten. Beim WAP-Pattern schreibt die Pipeline zuerst in eine isolierte Staging-Tabelle (`fact_video_metrics_wap`), führt alle Audits darauf aus und publiziert sie erst bei 0 Fehlern per Zero-Copy Table Clone (`CREATE OR REPLACE TABLE ... CLONE`) in die Produktion.

F18: Wie eliminiert man im Enterprise-Ausbau den Kopierschritt `load_to_bigquery.py` mittels BigQuery BigLake?

Indem PySpark die Apache Iceberg V2 Tabelle direkt in den GCS-Bucket (`gs://lakehouse-youtube-cloudanalytics-514018/iceberg/silver/videos`) schreibt. Über eine **BigQuery BigLake Iceberg Table** liest BigQuery die `vN.metadata.json` und Parquet-Dateien direkt auf GCS mit voller Spalten-Zugriffskontrolle und Snapshot-Isolation, ohne dass Daten physisch dupliziert oder hochgeladen werden müssen.

F19: Welche Schwachstelle hat `dbt snapshot` bei mehreren Mutationen desselben Videos innerhalb eines einzigen Micro-Batches?

`dbt snapshot` kann pro Aufruf für einen `unique_key` (`video_id`) nur genau einen neuen Zustand mit dem aktuellen Bestand vergleichen. Deshalb muss `dim_video_snapshot.sql` via `QUALIFY row_number() = 1` auf den jüngsten Satz pro Batch deduplizieren; Zwischenmutationen innerhalb desselben Batches gehen verloren. Um jede Einzelmutation eines Batches zu erfassen, berechnet man `valid_from` und `valid_to` (`LEAD(trending_date) OVER (...)`) direkt als deterministische Window-Function im dbt-Modell.

F20: Warum ist eine lokale `terraform.tfstate`-Datei im Team-Betrieb gefährlich und wie löst man das?

Liegt `terraform.tfstate` lokal auf einem Laptop, haben andere Teammitglieder und CI/CD-Runner keinen Zugriff auf den aktuellen Infrastruktur-State. Führen zwei Personen gleichzeitig `terraform apply` aus, kommt es zu Race-Conditions und Ressourcen-Korruption. Im Enterprise-Umfeld nutzt man ein `backend "gcs"` in einem versionierten GCS-Bucket, das über `.tflock`-Dateien automatisches State-Locking erzwingt.

F21: Wie sieht ein striktes Least-Privilege IAM-Konzept für diese Plattform aus?

Statt eines einzigen Sammel-Service-Accounts mit `*.admin`-Rollen trennt man nach dem Least-Privilege-Prinzip: 1. `sa-cicd-terraform` (nur für Infrastruktur-Provisionierung in GitHub Actions), 2. `sa-ingest-producer` (`roles/pubsub.publisher` nur auf den beiden Topics), 3. `sa-workflow-runtime` (`roles/bigquery.dataEditor` auf Dataset-Ebene + `roles/bigquery.jobUser`), 4. `sa-grafana-reader` (ausschließlich Read-Only `roles/monitoring.viewer`).

F22: Wie behandelt das Star-Schema Late-Arriving Facts oder Early-Arriving Facts (Late-Arriving Dimensions)?

Trifft ein Fakt ein, dessen `trending_date` zeitlich vor dem ersten `valid_from`-Zeitstempel der zugehörigen `dim_video`-Version liegt, würde ein strikter Point-in-Time `INNER JOIN` den Fakt verwerfen. Im Enterprise-DWH setzt man bei der initialen Version eines Videos `valid_from` auf ein technisches Startdatum (`1970-01-01`) oder nutzt einen `LEFT JOIN` mit Fallback auf einen Dummy-Dimensionssatz (`video_key = '-1'`, 'Unknown Version').

F23: Was ist das 'Small File Problem' in Apache Iceberg V2 bei hochfrequentem Streaming und wie behebt man es?

Schreibt ein Streaming-Consumer jede Minute eine kleine Parquet-Datei (z. B. 90 KB), entstehen tausende Kleinstdateien und Manifeste. Abfragen verlangsamen sich durch hohen Metadaten- und I/O-Overhead drastisch. Iceberg löst dies durch periodische Hintergrund-Compaction: `CALL local.system.rewrite_data_files(table => 'silver.videos', options => map('target-file-size-bytes', '134217728'))` bündelt kleine Dateien atomar zu 128-MB-Dateien, ohne laufende Leser zu blockieren.

F24: Wie erzwingt man Schema-Validierung bereits beim Senden an Google Cloud Pub/Sub (Schema-on-Write at the Edge)?

Anstatt fehlerhafte Datentypen erst im Consumer oder PySpark zu erkennen, definiert man in Terraform eine `google_pubsub_schema`-Ressource (im **Apache Avro** oder **Protocol Buffers** Format) und bindet diese an `topic-youtube-ingest`. Pub/Sub lehnt dann jede Nachricht, bei der `views` kein `long` ist, bereits am API-Gateway synchron mit `HTTP 400 InvalidArgument` ab.

F25: Wie sieht das Disaster-Recovery-Konzept (RTO / RPO) für diese Plattform aus?

• **Infrastruktur (RTO < 2 Minuten):** Durch Terraform IaC (`run_infra.ps1 apply`) lassen sich alle 7 GCP-Ressourcen in einer neuen Region sofort wiederherstellen. • **Streaming-Puffer (RPO = 0 bei Ausfall < 7 Tage):** Die Pub/Sub Subscriptions (`sub-youtube-ingest` / `sub-youtube-dlq`) speichern unbestätigte Nachrichten 7 Tage lang (`message_retention_duration = 604800s`). • **Data Warehouse & Lakehouse:** BigQuery bietet 7-Tage Time-Travel (`FOR SYSTEM_TIME AS OF`), der GCS-Bucket hat Object Versioning aktiviert, und Apache Iceberg V2 erlaubt Rollbacks auf jeden historischen Snapshot.

15. FinOps, Kosten-Engineering & Always-Free-Tier Kapazitätsplanung

Kostenoptimierung (**FinOps**) ist eine Kernverantwortung moderner Data Architects. Die gesamte YouTube DataOps Plattform (`v2.1.0`) ist so konzipiert, dass sie dauerhaft innerhalb des **Google Cloud Always Free Tiers** und **Grafana Cloud Free Tiers** betrieben werden kann.

15.1 Nachweis der 0,00 € Betriebskosten (Free Tier Kontingente vs. Verbrauch)

Cloud-Dienst	Monatliches Free-Tier-Kontingent	Realer Projekt-Verbrauch	Effektive Kosten
Google BigQuery	10 GB aktiver Speicher + 1 TB Abfragevolumen pro Monat frei.	< 5 MB Speicher (444 Zeilen) / < 100 MB Queries pro Monat.	**0,00 € / Monat**
Cloud Storage (GCS)	5 GB Standard-Storage frei; minimale Gebühren in EU bei Kleinstmengen.	Bucket `lakehouse-youtube-...` (< 10 MB; via `destroy` jederzeit löschbar).	**0,00 € / Monat**
Cloud Pub/Sub	10 GB Nachrichtenvolumen pro Monat kostenlos.	< 5 MB für Topics (`topic-youtube-ingest/dlq`) & Subscriptions (`sub-youtube-*`).	**0,00 € / Monat**
Cloud Workflows	5.000 interne Workflow-Schritte pro Monat kostenlos.	8 Schritte pro Workflow-Lauf (~240 Schritte bei täglichem Lauf).	**0,00 € / Monat**
Cloud Monitoring	Alle Standard-GCP-Dienstmetriken (`workflows`, `bigquery`, `pubsub`) kostenlos.	Ausschließliche Nutzung kostenloser GCP-Systemmetriken.	**0,00 € / Monat**
Grafana Cloud	Free Forever Plan: 3 User, 10k Metriken, 50 GB Logs inklusive.	1 User, 1 Dashboard (`youtube_dataops_monitoring.json`).	**0,00 € / Monat**

15.2 Implementierte Cost Guardrails (Schutz vor Kostenfallen)

- **Terraform Teardown-Garantie (`run_infra.ps1 destroy`):** Durch `force_destroy = true` am GCS-Bucket und `delete_contents_on_destroy = true` am BigQuery-Dataset lassen sich sämtliche 7 Cloud-Ressourcen jederzeit mit einem einzigen Befehl rückstandslos entfernen.
- **Grafana FinOps-Schwellenwerte:** Panel 4 (`BigQuery Bytes Billed`) schlägt bei über 100 MB auf Orange und bei 1 GB auf Rot um, sodass teure Abfragen sofort sichtbar werden.
- **Verzicht auf 24/7-Cluster:** Weder für Streaming (Serverless Cloud Pub/Sub) noch für Compute (ephemerer Docker-PySpark), Orchestrierung (Serverless Cloud Workflows) oder Monitoring (Grafana Cloud Weg 1) laufen dauerhafte VMs oder Kubernetes-Cluster.

Schlusswort: Dieses Repertorium dokumentiert den vollständigen, verifizierten Stand der **YouTube DataOps & Analytics Platform (2026 Edition)** über alle drei Releases (`v1.0.0`, `v2.0.0`, `v2.1.0`). Alle beschriebenen Skripte, Modelle, Tests, Terraform-Ressourcen, Cloud Workflows und Dashboards stimmen zu 100% mit dem produktiven Code im Repository überein.